三万字长文：大语言模型从零入门

原创 腾讯程序员 腾讯技术工程 Oct 13, 2026, 3:00 AM 广东 听全文



作者：flyflywang

| 导语 从图灵的模仿游戏到香农的熵，从一个神经元到千亿参数的 Transformer。一个只会预测下一个词的程序，何以通向智能？

> 遍览诗书亿万行，唯将下字细思量。云山千里文中见，可识秋风一夜凉？

导读

这篇文章想用一条线，讲清楚 ChatGPT 这类大语言模型是怎么回事：**它在做什么、为什么能做到、怎么做到的、做不好什么。**

不需要机器学习背景。用到的数学只有加减乘除和对数，对数也会从“折半了几次”讲起。全文的例子尽量来自同一个猜数字游戏和同一句“他口渴了，于是喝了一杯____”，看懂一个例子，后面会一路用到。

每一章回答的问题：

章节	回答的问题
语言模型的本质	语言模型在做什么？凭什么和“智能”扯上关系？预测好不好，怎么用数字衡量？极限在哪里？（信息量、熵、交叉熵）
神经网络	用什么来做预测？几十亿个参数，怎么调到合适的值？
Transformer	具体用什么结构，才能看懂整段前文？
从模型到 ChatGPT	有了概率，一段话是怎么写出来的？只会续写的模型，怎么变成聊天助手？它怎么联网、看图、用工具？模型为什么越做越大？它做不好什么？

怎么读：

- 全文近 3 万字，完整读完大约需要 1.5 小时，可以分两次读：前三章讲原理，最后一章讲从模型到产品。

- 带计算过程的代码块和公式，**跳过也不影响理解**，只看加粗的结论就能读通。想真正弄懂的话，建议拿笔跟着算一遍，每个数字都能核对。
- 全文大致沿着一串问题往下走，很多小节的结尾会抛出一个问题，由后面的内容来回答。读不下去的时候，回头找找前面留下的问题，就能找回主线。
- 只关心怎么用、有什么局限的读者，可以先读「语言模型的本质」的前两节，再直接跳到最后一章「从模型到 ChatGPT」。
- 最后的「总结」用十几行把全文串了起来，读完可以回头对照一下；末尾还留了几个问题供思考。

语言模型的本质

小龙在之前的年会上给大家分享过一段话：“信息就是惊喜，就是你猜不到的程度。信息量是由内容的离谱程度来决定的，越离谱信息量就越高。”这句话中暗含了语言模型的本质原理。

预测下一个 token

在解释之前，我们先玩一个词语接龙游戏，请补全：

```
床前明月____
```

所有人几乎都会填“光”，那再来看：

```
今天天气很____
```

这次答案不唯一：好、热、冷、不错……但也不是毫无规律，很少有人会填一个跟天气八竿子打不着的词，比如“今天天气很贵”。如果把各种可能按把握大小写下来，大概是这样（示意）：

候选	概率（示意）
好	35%
热	20%
冷	15%
不错	10%
其他所有候选合计	20%

如果把这道题交给语言模型来做，它给出的也是这样一张表。注意表里的“不错”是两个字，却和“好”一样算作一个候选。这是因为语言模型处理文字的最小单位不是“字”，而是 **token**。一个 token 可能是一个字，也可能是由多个字组成的一个词。以“今天天气很好”为例，模型看到的可能是这样：

```
今天 | 天气 | 很 | 好
```

这 6 个字被切成了 4 个 token：“今天”“天气”各占一个 token，“很”“好”各自单独成一个 token。所以模型“接龙”时，每一步补上的是一个 token，而不一定是一个字。

之所以要合并，一是省步数：模型每生成一个 token 都要完整算一遍，合并后“今天天气很好”只需算 4 步而不是 6 步。二是常一起出现的字本来就该当作整体，比如“今”后面经常跟着“天”，合成一个 token 后模型就不用反复去猜这种显而易见的搭配。

现在回到上面那张概率表，它就是语言模型做的事：

> **语言模型，是一个给定前文、输出“下一个 token 概率分布”的程序。**

模型每次只预测一个 token，那一整段话是怎么写出来的？做法很简单：从概率分布里选出一个 token 接到前文后面，再把新的前文交给模型预测下一个，如此循环（示意）：

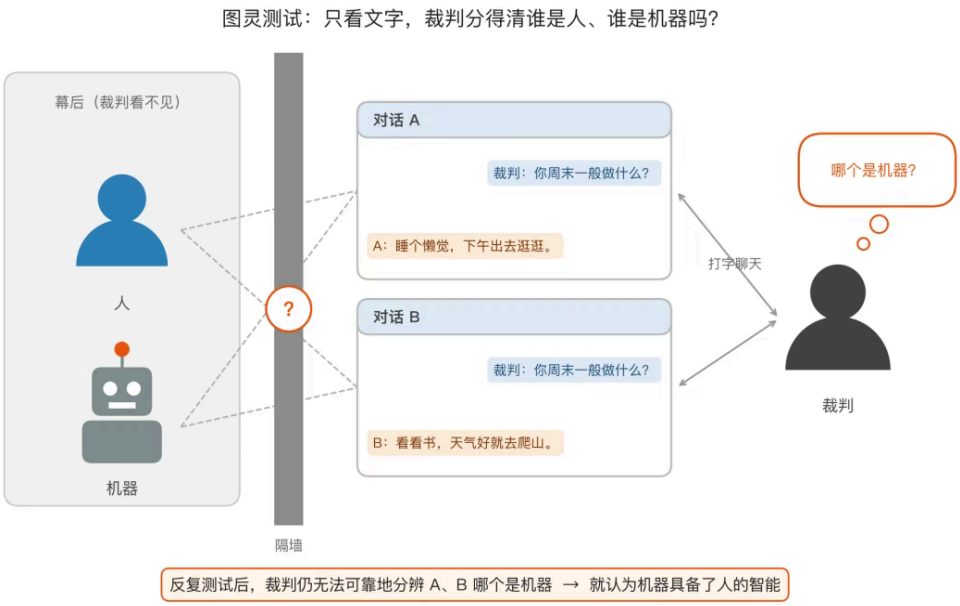
```
今天天气很          → 好
今天天气很好        → ，
今天天气很好，      → 适合
今天天气很好，适合  → 出门
```

为什么预测 token 能带来智能

“预测下一个 token”听起来像高级版的输入法联想，它凭什么和“智能”扯上关系？

1950 年，图灵在论文《计算机器与智能》里讨论“机器能思考吗？”。他觉得“思考”太难定义，于是换了个问法，设计了一个“模仿游戏”，也就是后来所说的**图灵测试**：[1]

- 一位裁判通过打字，和两个看不见的对象聊天；
- 其中一个是人，另一个是机器；
- 裁判可以问任何问题，聊完后判断哪个是人、哪个是机器。



如果裁判反复测试，也没法可靠地分辨出谁是机器，那就可以认为这台机器具备了人的智能。

> **分不清机器写的文字和人写的文字，就可以认为机器有了人的智能。**

注意，这个测试里裁判能看到的**只有文字**。机器用什么做的、内部怎么运转，都不重要，重要的是它写出来的东西像不像人写的。

再看语言模型做的事：它从海量人类文字中学习“给定前文，人接下来会写什么”。学得越准，它一个 token 一个 token 接出来的文字就越像人写的。如果它预测的概率分布和人写作时的分布完全一致，那它写出的文字和人写的就没有区别，裁判怎么看也分不出来。

所以，把预测下一个 token 做到极致，就是在通过图灵测试。

而要预测得准，光记住字词搭配是不够的。看几个例子：

```
法国的首都是____
```

要填“巴黎”，得有**知识**。

```
小明比小红高，小红比小刚高，三个人里最矮的是____
```

要填“小刚”，得会**推理**。

```
他打开冰箱，发现牛奶已经过期了，于是皱着眉头把它____
```

要填“倒掉”或“扔了”，得懂**常识**和人的行为。

人写下的文字里，藏着知识、逻辑、常识和情感。要把人接下来会写的 token 猜准，模型就不得不把这些东西学进去。**“预测”这个看似简单的目标，逼着模型去掌握文字背后的东西。**

当然，“分辨不出”只是图灵给出的一个行为标准，机器是否真的“理解”，至今仍有争论。但它给了我们一个清晰、可操作的目标：**让模型预测的概率尽量接近人。**

那“接近”该怎么衡量？预测得准不准，能用什么数字来表示？这就要从概率和信息的关系说起。

概率对应了多少的信息？

我从 **1～8** 中等概率地选一个整数，不告诉你。你可以问问题，但我只能回答“是”或“否”。

你当然可以挨个问：

```
是 1 吗？不是。
是 2 吗？不是。
是 3 吗？不是。
……
```

但这种问法容易浪费机会：每次得到“不是”，只排除一个数字。

更好的方式是，每次用一个问题把剩余候选分成数量相等的两组。假设我选的是 **6**：

第几问	你的问题	我的回答	剩余候选
开始	—	—	1、2、3、4、5、6、7、8
第一问	大于 4 吗?	是	5、6、7、8
第二问	大于 6 吗?	否	5、6
第三问	大于 5 吗?	是	6

三个问题，确定答案。

这不是因为你碰巧猜中了 6，而是因为这种问法能够让**任何一个答案**都在三次问答后被确定。

每得到一次回答，你都排除了一半候选：

```
8 个候选 → 4 个候选 → 2 个候选 → 1 个答案
```

信息，就是帮助你消除不确定性的东西。

在这个等概率游戏里，每次折半的回答提供 1 bit 信息，三次一共 3 bit。不要把 bit 想得太神秘，这里可以把它理解为：**区分两个等概率可能性所需要的信息量。**

再换几个范围试试：

- 从 4 个等概率数字中确定一个，需要折半 2 次。
- 从 8 个等概率数字中确定一个，需要折半 3 次。
- 从 16 个等概率数字中确定一个，需要折半 4 次。

候选越多，起初越不确定；要找到答案，需要获得的信息也越多。

顺带说一句：折半问法的好，不只是“最多三次”肯定能问出来，还在于**它在平均意义上也是最省问题的**。

原因很直接：每次“是 / 否”回答最多只带来 1 bit 信息，而确定 8 个等可能候选中的一个至少需要 $\log_2 8=3$ bit，所以**任何问法平均都至少要问 3 次**。折半问法每次都把候选对半分、每次都拿满 1 bit，平均下来正好 3 次，刚好踩在这个下限上。

到这里，就可以把“概率”和“信息量”直接对上号。**单个结果的信息量**是：

$$I(x)=\log_{1/2}p(x)$$

为什么底数是 1/2？因为信息量本质上就是“折半了多少次”。概率为 1/2 的结果，相当于在一半的可能里确定它，折半 1 次，对应 1 bit；概率为 1/8 的结果，要在八分之一的可能里确定它，折半 3 次，对应 3 bit。$\log_{1/2}$ 正好把这个“折半次数”数出来：概率每缩小一半，对数就加 1。

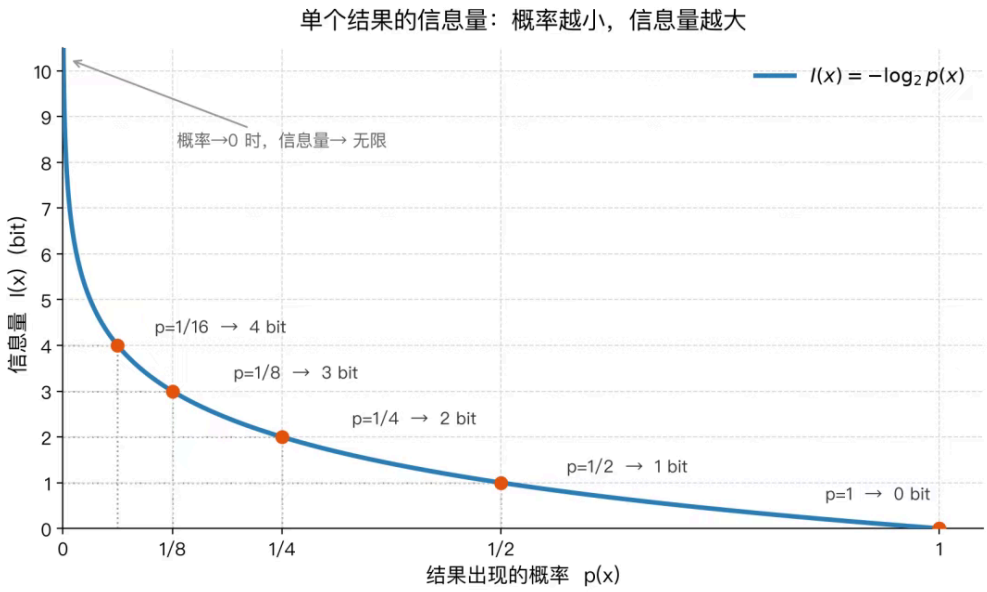
你只需要记住三点：

- `p(x)`
  是结果出现的概率。
- 概率越小，结果出现时带来的信息量越大。
- 如果一个结果本来就百分之百确定（概率为 1），知道它发生并不会带来新信息，信息量为 0。

第二点，就是开头小龙那句话：“信息就是惊喜，就是你猜不到的程度……越离谱信息量就越高。”**越离谱，就是概率越小；概率越小，信息量越大。** 听到“明天太阳从东边升起”，你毫无惊喜，因为它的概率接近 1，信息量接近 0；听到“明天太阳从西边升起”，那才是天大的新闻。

不过一般书面上更常见的写法，是把它写成负的以 2 为底。这两种写法其实是同一个式子（后面都采用这种写法）：

$$I(x)=-\log_2 p(x)$$



熵：平均而言，答案有多难确定？

现在改变规则：我选 1 的概率是 90%，剩下 10% 的概率分给 2～8。

这时，“大于 4 吗”未必还是最有效的第一问。你可以先问：

> 是 1 吗？

90% 的时候，一问就结束了。

这说明，难度不只由“有几个候选”决定，还与**各个候选出现的概率**有关。同样是 8 个数字：

- 如果每个数字都一样可能，你很难提前判断。
- 如果几乎每次都是 1，你已经知道很多，剩余的不确定性就小。

要描述游戏的平均难度，就得把每种结果的信息量，按照出现概率加权平均。这个平均值就是**熵**，它衡量的是这种平均不确定性。[2]

$$H(X)=\Sigma_x p(x)I(x)=-\Sigma_x p(x)\log_2 p(x)$$

用硬币对比一下：

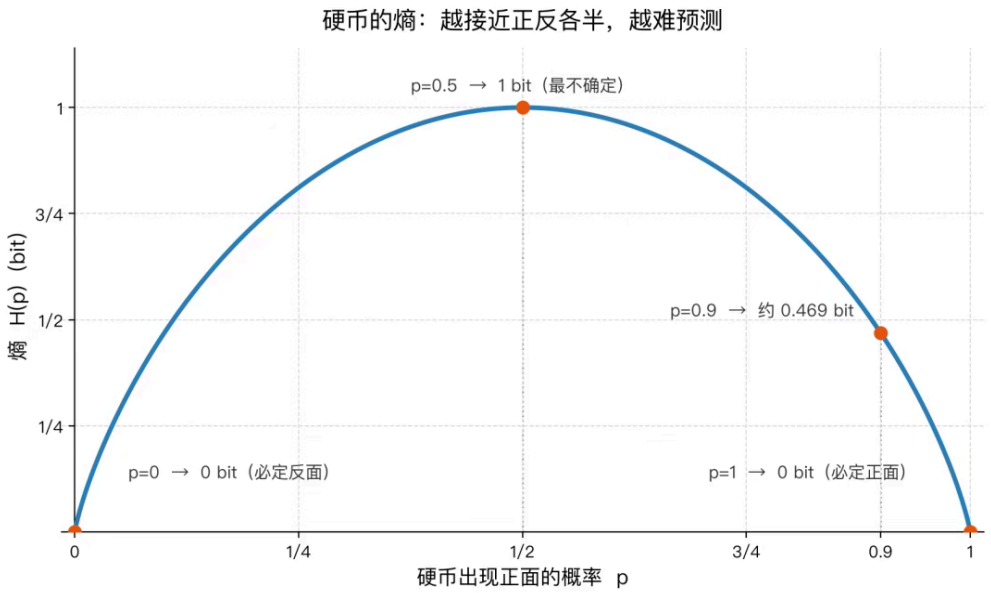
硬币的情况	熵	为什么
正反各 50%	1 bit	两种结果同样可能，不好提前判断
正面 90%，反面 10%	约 0.469 bit	大部分时候可以预期是正面
必定正面	0 bit	没有不确定性

第二行是这样算的：

```
正面：概率 0.9，信息量 -log₂0.9 ≈ 0.152 bit
反面：概率 0.1，信息量 -log₂0.1 ≈ 3.322 bit
熵 = 0.9 × 0.152 + 0.1 × 3.322 ≈ 0.137 + 0.332 ≈ 0.469 bit
```

罕见的反面信息量很大（3.3 bit），但它只占 10%，拉不高平均值。

把正面概率从 0 连续变到 1，就能看到熵的整体形状：正反各半时最不确定（1 bit），越偏向某一面，熵越低，到必定正面或必定反面时降为 0。



回到猜数字游戏，用同样的方法算一下两种规则下的熵。

1～8 等概率：每个数字的概率都是 1/8。

```
每个数字：概率 1/8，信息量 -log₂(1/8) = 3 bit
熵 = 8 × (1/8 × 3) = 3 bit
```

正好对应折半问法需要的 3 个问题。

1 的概率为 90%：剩下的 10% 平分给 2～8，每个数字的概率是 0.1 / 7 = 1/70 ≈ 0.0143。

```
数字 1：概率 0.9，信息量 −log₂0.9 ≈ 0.152 bit
数字 2～8：每个概率 1/70，信息量 −log₂(1/70) ≈ 6.129 bit
熵 = 0.9 × 0.152 + 7 × (1/70 × 6.129)
  ≈ 0.137 + 0.613
  ≈ 0.750 bit
```

猜数字的情况	熵
1～8 等概率	3 bit
1 占 90%，2～8 平分 10%	约 0.750 bit

同样是 8 个候选，概率一旦集中到 1 上，平均不确定性就从 3 bit 降到约 0.75 bit，只剩原来的四分之一左右。

熵不只是一个“难度分数”，它还有一个很实在的含义：平均最少要用多少 bit 才能把答案记下来，也就是数据能被压缩到的极限，任何无损编码的平均长度都不可能低于熵。[3]

规律掌握得越准，编码就越短；能压到熵，说明数据里所有能被利用的规律都已经挖出来了，剩下的只是真正无法预测的随机性。

语言也是一样。下一个 token 的真实概率里，藏着语法、常识、逻辑和人的意图。模型预测得越准，就越能把文字压缩到接近它的熵；而要做到这一点，就不得不把文字背后这些深层的规律学进去。这就是 **「压缩即智能」** 的核心思路：**能把数据压缩得越短，说明对数据背后规律的掌握越深。**[4]

这和前面的图灵测试，是同一件事的两面：

- 图灵测试给出的是**标准**：预测得和人一样，就可以认为有了人的智能。
- 压缩即智能给出的是**原因**：要预测得和人一样，就得把文字压缩到接近它的熵；而要压得这么短，最有效的办法就是学会文字背后的规律。

不过要注意，“预测越准、压缩越短”是数学事实；“压缩得好就是智能”只是一个有启发的研究视角，而不是已被证明的定理。比如“一万个 A”可以写成“A 重复一万次”，一个简单程序就能压得极短，但不能因此说它会推理。

为什么“压得短”意味着“学到了规律”，而不是把文字死记硬背下来？这个问题留到后面「背答案还是学方法」一节再细说。

那么，如果模型对概率的估计不准，编码会比熵多花多少？

交叉熵

先说清楚一点：熵是数据本身的性质。只要我选数字的规则不变，熵就一直是 0.75 bit，模型估计得准不准都改变不了它。会变的是：**按模型的估计去编码时，实际要花多少 bit。**

假设真实规则还是“1 占 90%，2～8 平分 10%”，但模型以为 8 个数字等概率。用 $p(x)$ 表示真实概率，$q(x)$ 表示模型认为的概率。模型会按自己的判断安排编码：认为谁常见，就给谁短编码，结果 x 的码长就是模型眼中的信息量 $-\log_2 q(x)$。

模型认为每个数字都是 1/8，于是给每个数字都分配 3 bit。可实际上 90% 的时候出现的都是 1，本来 0.152 bit 就够了，却每次都花了 3 bit：

```
数字 1：真实出现 90%，模型给的码长 -log₂(1/8) = 3 bit
数字 2～8：真实一共出现 10%，模型给的码长也都是 3 bit
平均码长 = 0.9 × 3 + 0.1 × 3 = 3 bit
```

比熵多花了 2.25 bit，这些都是估计错了概率而白白浪费的。

把这个算法写成公式，就是**交叉熵**：

$$H(p,q)=-\Sigma_x p(x)\log_2 q(x)$$

和熵的公式相比，只是把对数里的 $p(x)$ 换成了 $q(x)$：

- $p(x)$
  （真实概率）负责**加权**：这个结果实际多常出现。
- $q(x)$
  （模型概率）负责**定价**：这个结果每出现一次要花多少 bit。

当模型的估计和真实完全一样（$q=p$）时，交叉熵就等于熵。

再换几种模型对比一下（真实规则都是 1 占 90%）：

模型认为的概率	交叉熵	比熵多花
和真实一样：1 占 90%	约 0.75 bit	0
等概率（什么规律都不知道）	3 bit	2.25 bit
2 占 90%，其余平分（自信地猜错）	约 6.04 bit	约 5.29 bit

从这张表可以看出两点：

- **交叉熵永远不低于熵**
  ，只有模型的概率和真实概率完全一致时才相等。模型估计得越偏，多花的就越多。
- **自信地猜错，比什么都不知道还糟。**
  等概率的模型每次都花 3 bit；认定是 2 的模型，在真正的 1 出现时（90% 的情况），每次要花约 6.13 bit。

不过，交叉熵公式里要用到真实概率 $p(x)$，可人写作时的真实概率，谁也不知道。好在并不需要知道：**真实概率已经体现在真实数据里了。**

把猜数字玩 1000 局，记下每局真实出现的数字，再把模型给这个数字的码长 $-\log_2 q(x)$ 取平均。1 大约出现 900 次，2～8 一共大约出现 100 次，平均的时候自然就按真实概率加了权，结果会接近交叉熵。

语言模型也是这样算的。拿一段人写的真实文本，在每个位置上看模型给**真实下一个 token** 的概率：

```
今天天气很____      真实下一个 token：好
模型给"好"35%    → -log₂0.35 ≈ 1.51 bit
模型给"好"5%     → -log₂0.05 ≈ 4.32 bit
```

把海量文本里每个位置的值取平均，就是模型在这批文本上的交叉熵。它就是训练语言模型时用的**损失函数**，越低说明模型预测得越准。

交叉熵还有一种更直观的说法，叫**困惑度**，等于 $2^{\text{交叉熵}}$，意思是：模型的犹豫程度，相当于在几个等可能的候选里猜一个。比如等概率模型的交叉熵是 3 bit，困惑度就是 $2^3=8$，正好对应"从 8 个数字里猜"；掌握了规律的模型（约 0.75 bit），困惑度只有约 1.7，相当于只在不到 2 个候选里犹豫。

实际训练中，对数通常以 e 为底，单位叫 nat，和 bit 只差一个固定倍数（1 nat ≈ 1.44 bit），不影响结论。

到这里，前面的问题就有了答案：要"让模型预测的概率尽量接近人"，"接近"就用交叉熵来衡量。

```
交叉熵越低
= 模型的概率越接近人写作时的真实概率
= 用模型压缩文本越短，越接近文本本身的熵
```

训练语言模型，就是在海量的人类文本上，不断降低交叉熵。

香农的实验

交叉熵告诉了我们怎么给模型打分。但还有一个问题：人类语言本身的熵大概是多少？模型的交叉熵最低能降到哪里？

1951 年，香农在论文《印刷英语的预测与熵》里估算了这个数。[5] 英文有 26 个字母，加上空格，一共 27 种符号；已知的信息越多，下一个字符越好猜：

已知的信息	每个字符的熵（约）
什么都不知道，27 种符号等可能	4.75 bit
知道每个字母各自出现的频率	4.03 bit
知道前 1 个字母	3.32 bit
知道前 2 个字母	3.1 bit

已知的信息	每个字符的熵（约）
知道前面约 100 个字母	0.6～1.3 bit

第一行就是“等概率猜数字”（$\log_2 27 \approx 4.75$）；第二行像“1 占 90%”那一局：E 比 Z 常见得多。从第三行开始用上了**上下文**：看到 Q，下一个几乎一定是 U；中文里看到“床前明月”，下一个字几乎是确定的。

> **已知的上下文越多，下一个字越好猜，熵就越低。** 这也是语言模型要尽可能看长前文的原因。

前几行可以靠数数统计，最后一行不行：前文一长，组合多到数不过来。香农的办法是**直接让人当预测者**：遮住一段英文，让受试者一个字符一个字符地猜，记下每个字符是第几次猜中的。懂英语的人大多一次就中，论文的一次实验里，102 个字符有 79 个是第一次猜中的。

设想发送方和接收方各有一个一模一样的预测者：发送方只传这份“第几次猜中”的记录，接收方按同样的顺序去猜，就能还原原文。所以这份记录就是压缩后的文本。它里面满是 1，能压得很短；能压到多短，就反映了英文本身的熵。香农由此估算出：英文每个字符的熵只有 **0.6～1.3 bit**，大约相当于抛一次硬币。

七十多年前，香农用人脑当预测者；语言模型则是一个机器预测者，而且给出的不只是猜测的先后顺序，而是每个候选的概率。由于交叉熵永远不低于熵，**模型在真实文本上的交叉熵，就是语言熵的一个上界**：模型越好，这个上界压得越低，越接近语言真实的熵。

这个机器预测者，该用什么来做？

神经网络

最先想到的办法，可能是让人来写规则：“口渴”后面接“水”，“困了”后面接“咖啡”，主语后面接动词……可语言里的规律实在太多，搭配、语法、常识、语气，还处处有例外，靠人一条条写，永远写不完，也写不对。早年的机器翻译走过这条路，最后都卡在这里。

规则只能让机器自己从海量文本里统计和学习出来。

最简单的统计办法是：把海量文本翻一遍，数一数某段前文后面都跟过什么。比如“一杯”后面跟“水”出现过 600 次、跟“茶”300 次、跟“咖啡”100 次，那下次再看到“一杯”，就预测“水”60%、“茶”30%、“咖啡”10%。把各种前文的统计结果记成一张大表，用的时候去查，早期的语言模型基本就是这么做的。

可前文一长，这张表就建不出来：词表有 10 万个 token，只看前 10 个，可能的组合就有 10 万的 10 次方，也就是 10^{50} 种，绝大多数从没在文本里出现过。比如“他在沙漠里走了三天三夜，终于喝到了一杯____”，文本里很可能找不到一模一样的句子，查表只能得到“没见过”；人却一眼就能看出，这和“口渴了想喝水”是同一类情境。

表的问题在于只会死记：见过的前文能查到，没见过的就毫无办法。**我们需要的是像人一样能举一反三：从见过的句子里学到规律，再用到没见过的句子上。** 神经网络就是这样一种工

具。

下面先看神经网络长什么样，再看它怎么学，最后看怎么判断它是真学会了，还是只是背下来了。

神经元

神经网络的基本单元叫**神经元**。先看一个最简单的神经元：只有一个输入 x，一个输出 y。它做两件事：

1. 算一个一次函数 $wx+b$。w 叫**权重**，决定输入对结果的影响有多大、是正还是负；b 叫**偏置**，是一个固定的加减量。
2. 把结果"弯折"一下，也就是送进一个非线性函数。最常用的叫 **ReLU**：负数变成 0，正数保持不变，即 $\mathrm{ReLU}(z)=\max(0,z)$。

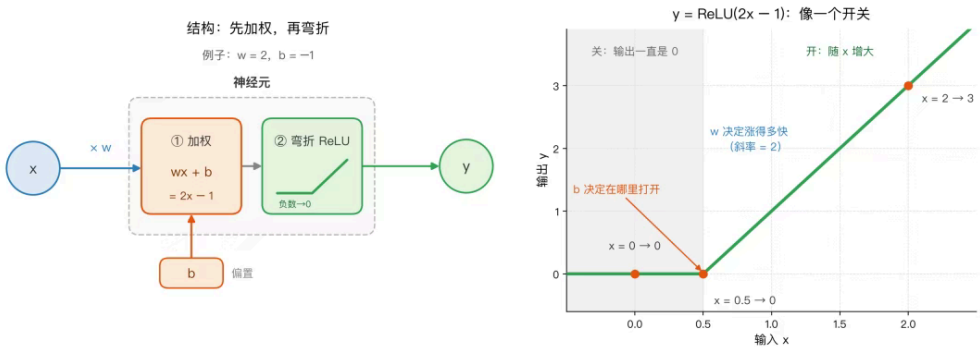
合起来，一个神经元就是：

$$y=\mathrm{ReLU}(wx+b)$$

举个例子，取 $w=2$、$b=-1$：

```
x = 0:    2 × 0   − 1 = −1   → ReLU(−1) = 0
x = 0.5:  2 × 0.5 − 1 = 0    → ReLU(0)  = 0
x = 2:    2 × 2   − 1 = 3    → ReLU(3)  = 3
```

x 小于 0.5 时，输出一直是 0；超过 0.5 之后，才开始随 x 增大。所以可以把神经元想象成一个**开关**：b 决定它在哪里打开（这里是 $x=0.5$），w 决定打开之后增长得多快。



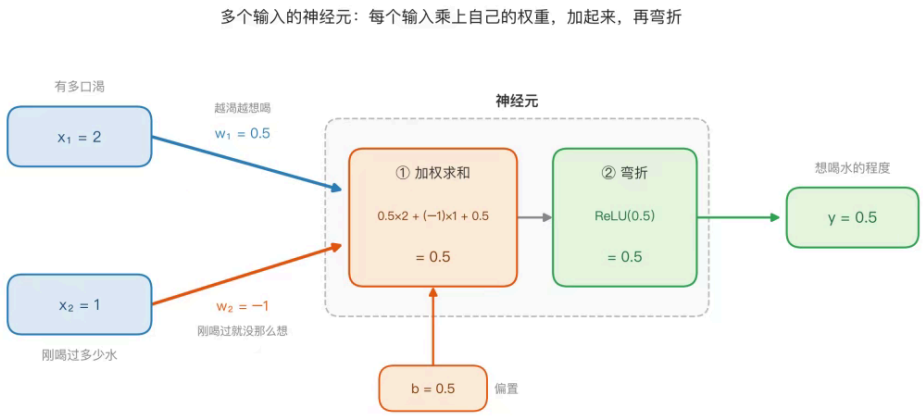
真实的神经元有很多个输入。 一个 token 会被表示成一串几千个数字，这样一串数字叫**向量**（Transformer 一章的「嵌入」一节会细讲）；上一层也有几千个神经元，每个都产生一个输出。所以一个神经元面对的通常不是一个 x，而是一组 $x_1, x_2, \cdots, x_n$。做法是给每个输入配一个自己的权重，乘完加起来，这就是**加权求和**：

$$y=\mathrm{ReLU}(w_1x_1+w_2x_2+\cdots+w_nx_n+b)$$

只有一个输入时，它就退回到 $\mathrm{ReLU}(wx+b)$。举个两个输入的例子（示意，真实神经元的含义通常没这么好解释）：x_1 表示"有多口渴"，x_2 表示"刚喝过多少水"，这个神经元衡量"想

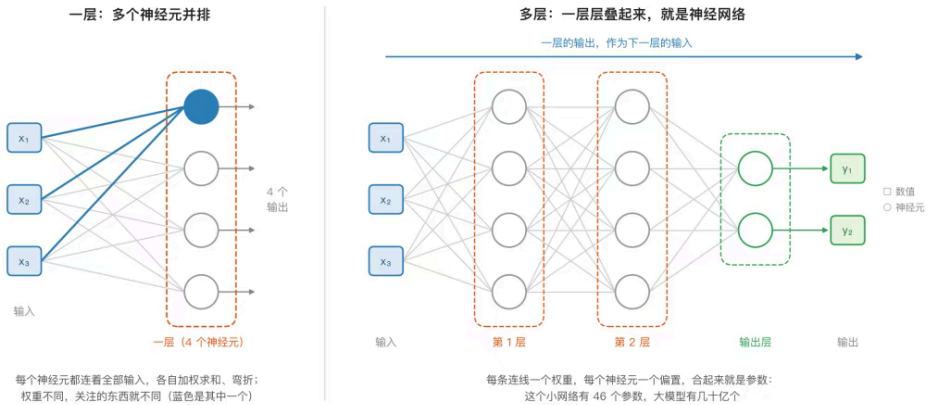
喝水的程度”：

```
输入：   x₁ = 2（挺渴的）          x₂ = 1（刚喝过一点）
权重：   w₁ = 0.5（越渴越想喝）    w₂ = −1（刚喝过就没那么想）
偏置：   b = 0.5
加权求和：0.5 × 2 + (−1) × 1 + 0.5 = 0.5
弯折：     ReLU(0.5) = 0.5
```



为什么叫“神经元”？ 名字借自大脑：大脑里的神经元接收许多其他神经元传来的信号，汇总起来超过某个阈值才会放电。加权求和对应汇总，ReLU 对应“过了阈值才放电”。1943 年 McCulloch 和 Pitts 按这个思路提出了最早的神经元数学模型，名字就沿用了下来。[6] 不过只是借个名字，今天的神经网络并不追求像大脑。

很多个神经元并排是一**层**，一层的输出作为下一层的输入，一层层叠起来就是神经网络。所有的权重和偏置统称**参数**，它们决定了这个网络算的是什么函数。所谓“训练”，就是去调这些参数。



为什么一定要弯折？ 因为一次函数套一次函数，还是一次函数：比如先算 $z=2x+1$，再算 $3z-2$，结果是 $6x+1$。如果只有加权求和，叠多少层都等价于一层，只能表示直线式的简单关系。加上 ReLU 这样的弯折，网络才能表示弯弯曲曲的复杂关系。

Softmax：从分数到概率

网络搭好了，可它算出来的只是一堆数字，而语言模型要给出的是下一个 token 的概率。

语言模型的最后一层比较特殊：词表里的每个 token 都对应一个神经元，这个神经元算出的加权求和 $w_1x_1+\cdots+w_nx_n+b$，就是这个 token 的**分数**（常称为 logits）。最后一层不再做 ReLU 弯折，否则所有负分都会变成 0，分不出高低。

但分数还不能直接当概率用：

- 加权求和的结果可正可负，而概率不能是负数；
- 所有 token 的分数加起来，也不等于 1。

所以还需要 **Softmax** 把分数变成概率：先对每个分数取指数（保证都是正数，而且分数越高、指数越大），再除以总和（保证加起来等于 1）。

$$q_i=(e^{s_i})/(\Sigma_j e^{s_j})$$

假设只有三个候选（示意）：

候选	分数	取指数	概率
水	3.0	约 20.09	约 69%
茶	2.0	约 7.39	约 25%
书	0.5	约 1.65	约 6%

分数越高，概率越大；而且每个候选的概率都大于 0，即使模型猜错了，交叉熵也不会变成无穷大。

到这里，语言模型的骨架就有了：

```
前文 → 神经网络（很多层"加权求和 + 弯折"）→ 每个 token 的分数 → Softmax → 下一个 token 的概率
```

接下来的问题是：这样一层层叠起来的网络，能力上限在哪里？

通用逼近定理

答案出奇地好：**只要神经元足够多，哪怕只有一层，神经网络也能以任意精度逼近任意连续函数。** 这就是**通用逼近定理**（Universal Approximation Theorem），1989 年前后由 Cybenko、Hornik 等人证明，后来又被推广到 ReLU 这类激活函数上。[7][8][9]

听起来很抽象，但用 ReLU 可以很直观地看出为什么。

第一步：3 个 ReLU 拼出一个"小帐篷"。

$$\mathrm{ReLU}(x)-2\mathrm{ReLU}(x-1)+\mathrm{ReLU}(x-2)$$

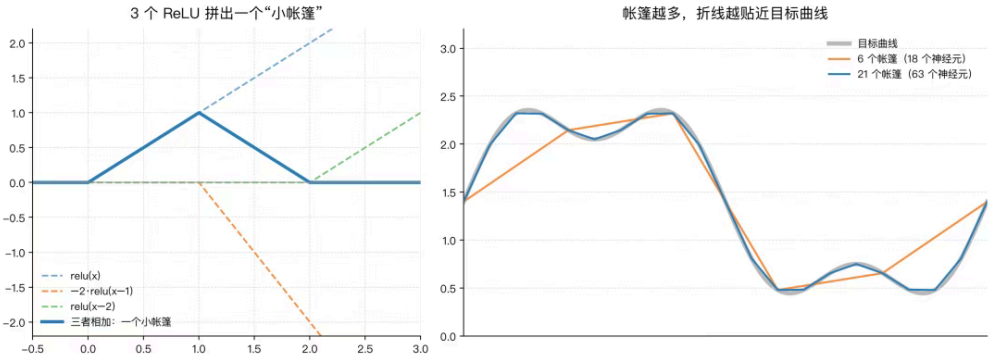
逐段看：

```
x ≤ 0:        三项都是 0                    → 0
0 ≤ x ≤ 1:    只有第一项起作用              → x，从 0 升到 1
```

```
1 ≤ x ≤ 2:   x − 2(x − 1) = 2 − x          → 从 1 降到 0
x ≥ 2:       x − 2(x − 1) + (x − 2) = 0    → 0
```

结果是一个在 x = 1 处达到顶点、两边都为 0 的小帐篷。

第二步：把很多帐篷并排摆放。 每个帐篷的位置和宽窄由权重和偏置决定，高度由下一层的权重决定。按目标曲线在各处的高度调好每个帐篷，加起来就是一条贴着目标曲线的折线。帐篷越多、越窄，折线就越贴近。



“输入前文，输出下一个 token 的概率”也是一个函数，只不过输入和输出都是成千上万维的。通用逼近定理告诉我们：**神经网络有能力表示它。**

但这个定理有三件事没说：

- **没说要多少神经元。**
  在复杂的高维问题上，如果只用一层，需要的神经元可能多到天文数字。
- **没说参数怎么找。**
  它只保证好参数“存在”，不保证训练能找到。
- **没说能不能推广。**
  能拟合见过的数据，不等于能在没见过的数据上做对。

第一个问题，引出了“深度”；后两个问题，留到讲模型怎么学习时再回答。

深层网络的优势

通用逼近定理说一层就够，可实际的大模型都很深：GPT-3 有 96 层，Llama 3 的 80 亿参数版本也有 32 层。[10] [11] 为什么不用一层很宽的网络？

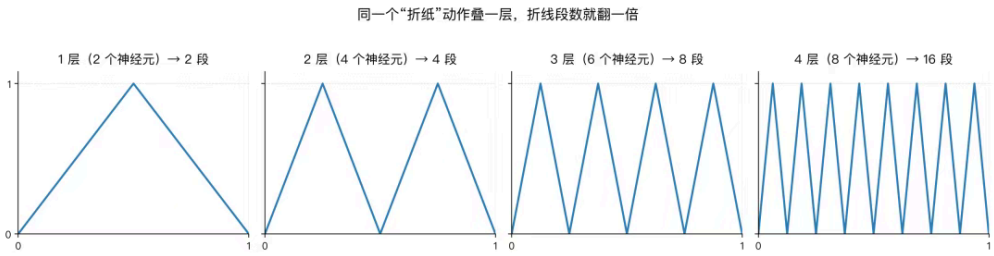
原因一：同样多的神经元，深网络能表示复杂得多的函数。

看一个“折纸”的例子。下面这个函数只用 2 个神经元，在 0～1 之间是一个尖顶：

$$t(x) = 2\mathrm{ReLU}(x) - 4\mathrm{ReLU}(x - 0.5)$$

x 从 0 走到 0.5，t(x) 从 0 升到 1；x 从 0.5 走到 1，t(x) 又从 1 降回 0。相当于把 0～1 这段线对折了一次。

把它的输出再送进一层同样的 t，就是再对折一次。每多一层，折线的段数就翻一倍：



层数	用掉的神经元	折线段数	只用一层，至少要多少神经元
4	8	16	15
10	20	1024	1023
20	40	约 105 万	约 105 万

差别在于：一层网络里，每个神经元只能贡献一个“拐点”，拐点只能一个一个地加；深网络每一层都在上一层的结果上**重复使用**同一个操作，拐点成倍增长。这个例子来自 Telgarsky 2016 年关于深度优势的研究。[12]

原因二：复杂的规律，本来就是一层层搭起来的。

理解一句话，也是逐层进行的：

```
字 → 词 → 短语 → 句子的意思 → 上下文中的意图
```

深网络可以分工：前面的层处理拼写、搭配这类表层模式，后面的层在前面结果的基础上，组合出语义、指代和推理。研究者在真实模型里也观察到了类似的趋势：较低的层更多捕捉表层模式，较高的层更多捕捉语义。[13]

这就像写程序：一层网络相当于把所有情况写进一张巨大的 if-else 表；深网络相当于先写一些小函数，再用小函数组合出大函数。**中间结果能被反复复用，所以省得多。**

代价：越深越难训练。

数值要一层层往后传，训练时调整参数的信号还要一层层往回传，层数一多，两个方向都容易出问题。2015 年之前，主流网络大多只有十几到二十几层，再深就很难训练好。

让几十上百层的网络也能稳定训练的，主要靠两项技术：**归一化**和**残差连接**。要看清它们为什么管用，得先知道网络是怎么学习的。

梯度下降

刚搭好的网络，参数都是随机的，输出的概率表也是乱的，交叉熵很高。训练要做的，就是**在海量文本上把交叉熵降下来。** 通用逼近定理只保证好参数“存在”，没说怎么找。问题是参数有几十亿个，每个该往哪个方向调、调多少？

先看只有一个参数的情况。模型是“输出 = w × 输入”，只有一条数据：输入 2，正确答案 6。损失用误差的平方（语言模型用交叉熵，道理相同）。

设 w 一开始是 1：

```
预测 = 1 × 2 = 2，误差 = 2 − 6 = −4，损失 = 16
```

把 w 调大一点点，从 1 调到 1.01：

```
预测 = 2.02，误差 = −3.98，损失 ≈ 15.84，下降了约 0.16
```

参数增加 0.01，损失减少约 0.16，变化的比例约为 −16，这个比例就叫**梯度**：参数往某个方向微微一动，损失会怎么变。梯度为负，说明调大参数损失会下降，应该调大；梯度为正，就应该调小。

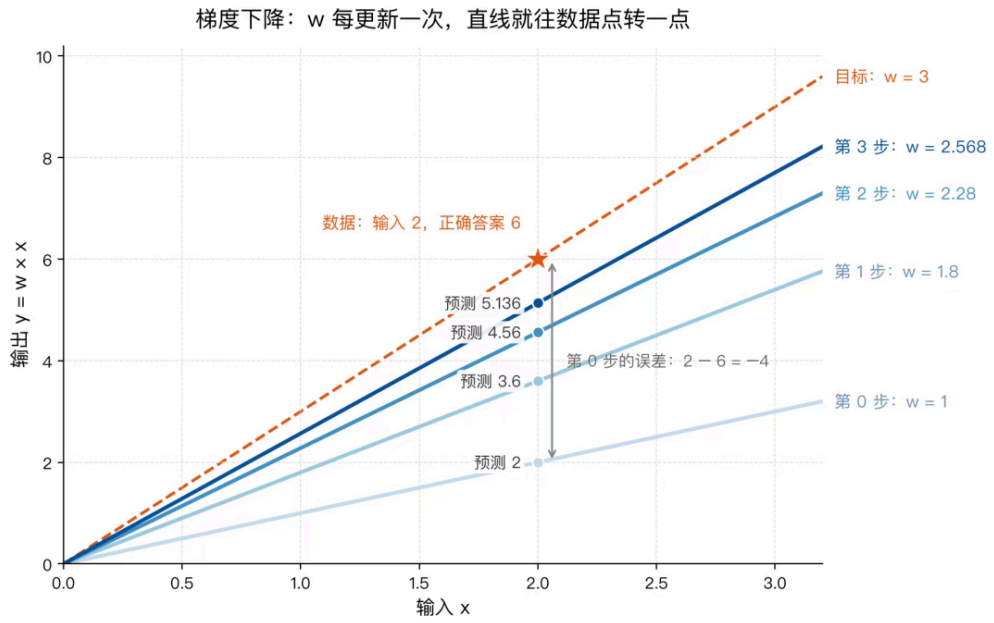
可以把损失想象成一片山地，参数是你站的位置，梯度告诉你脚下的坡朝哪边、有多陡。**看不到整座山，但每一步都往下坡走，就能走到谷底附近。** 这就是**梯度下降**：

```
新参数 = 旧参数 − 学习率 × 梯度
```

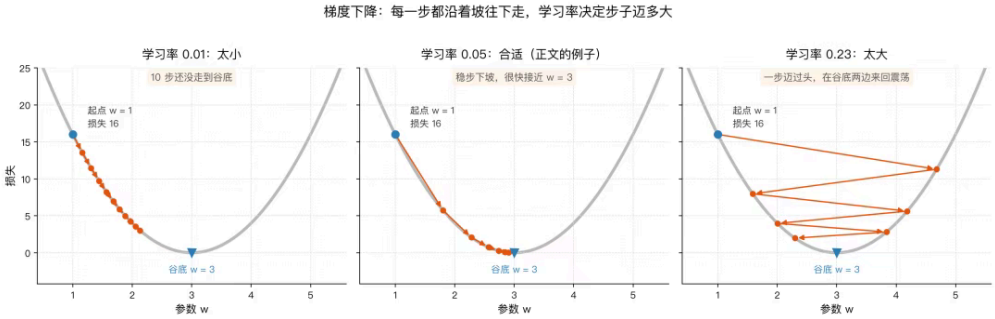
减号表示往梯度的反方向走，也就是下坡。**学习率**控制每一步迈多大。实际训练中，梯度不用一点点试，而是用微积分里的求导直接算出来，这里不展开。取学习率为 0.05，反复更新：

步数	w	预测	损失	梯度
0	1	2	16	−16
1	1.8	3.6	5.76	−9.6
2	2.28	4.56	约 2.07	−5.76
3	2.568	5.136	约 0.75	约 −3.46
……	越来越接近 3		越来越接近 0	

把每一步的 w 画成直线 y = wx，就能看到直线一步步转向数据点 (2, 6)，预测和正确答案的差距越来越小：



学习率太小，走得慢；太大，会一步迈过头，在谷底两边来回震荡。



这就是“参数怎么找”的答案。要说明的是，梯度下降只看脚下的坡，**不保证找到整片山地最低的那个谷底**，可能停在某个局部的小坑里。但实践表明，对参数极多的大网络，它找到的参数通常已经足够好。

语言模型的梯度：预测减去答案

Softmax 配上交叉熵，梯度有一个特别简洁的形式：

```
每个候选分数的梯度 = 模型给它的概率 − 实际答案（实际出现的记为 1，其他记为 0）
```

还是“水、茶、书”的例子（分数沿用上一节 Softmax 那张表），前文是“他口渴了，于是喝了一杯____”，真实答案是“水”：

候选	分数	预测概率	实际答案	梯度（概率 − 答案）	新分数（分数 − 梯度）
水	3.0	0.69	1	−0.31	3.00 − (−0.31) = 3.31
茶	2.0	0.25	0	+0.25	2.00 − 0.25 = 1.75
书	0.5	0.06	0	+0.06	0.50 − 0.06 = 0.44

用学习率 1，按“新参数 = 旧参数 − 学习率 × 梯度”更新一次，就得到最后一列的“新分数”（梯度为负，减掉它就等于加上，分数被拉高）。“水”的概率从 69% 升到约 79%，交叉熵

从约 0.54 bit 降到约 0.34 bit。

实际出现的 token 被往上拉，其他 token 按原本的概率被往下压。 模型原来越看好某个错误答案，它被压得越狠。

为什么纠错能变成学规律?

真实训练时，每次取一批文本，把所有位置的梯度平均起来，再更新一次参数，如此重复。GPT-3 最大的版本每批约 320 万个 token，一共更新了 9 万多次。[10]

关键在于：**所有句子的预测，共用同一套参数。** 每个句子的梯度都在把参数往"对自己更好"的方向推。只对个别句子有用的调整，会被其他句子的梯度抵消；对大量句子都有用的调整，比如"口渴和喝水有关"，会被反复加强。

> **一种内部计算方式，如果能在大量不同的句子上减少错误，就会在反复的梯度更新中被保留和强化。**

剩下的问题是：几十亿个参数，每个参数的梯度怎么算?

反向传播

对一个参数求导不难，难的是数量：大模型有几十亿个参数，一层套一层。如果每个参数单独去求，或者像前面那样逐个试探，每求一个都要把整个模型算一遍，根本算不完。

反向传播只需要正向算一遍、反向推一遍，就能得到所有参数的梯度。它利用的是微积分里的**链式法则**，1986 年由 Rumelhart、Hinton 等人推广开来。[18]

一个两层的例子

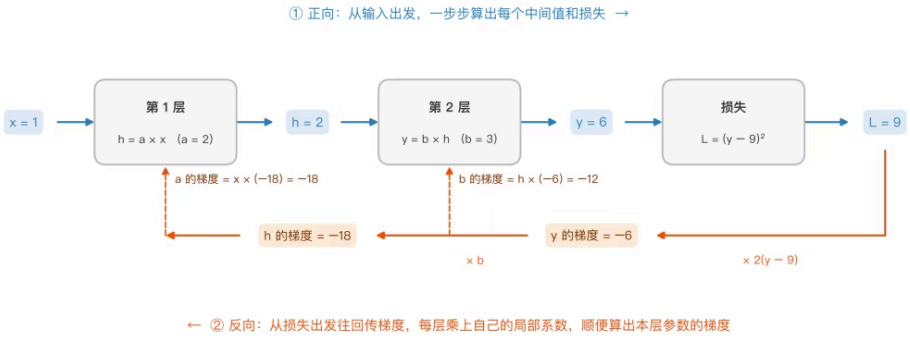
```
输入 x = 1
第 1 层：h = a × x        (参数 a = 2)   → h = 2
第 2 层：y = b × h        (参数 b = 3)   → y = 6
损失：    L = (y − 9)²                    → L = 9
```

现在从损失出发，**一层一层往回推**：

```
① 损失对 y 的梯度：2 × (y − 9) = −6
   （平方的求导结果。试探验证：y 从 6 调到 6.01，损失从 9 降到约 8.94，约为 −6 × 0.01）

② 第 2 层：y = b × h
   b 的梯度       = h × (−6) = 2 × (−6) = −12
   传给 h 的梯度 = b × (−6) = 3 × (−6) = −18

③ 第 1 层：h = a × x
   a 的梯度       = x × (−18) = 1 × (−18) = −18
```



可以用试探法验证：把 a 从 2 调到 2.01，损失从 9 降到约 8.82，下降约 0.18，正好是 −18 × 0.01。

注意每一层做的事：**拿到后面传回来的梯度，乘上自己这一层的局部系数，算出自己参数的梯度，再把梯度继续传给前一层。** 每一层只需要知道自己的输入和后面传来的梯度，不用了解整个网络。

直观地说，这就是**分摊责任**：最终结果错了，先看最后一层“贡献了多少错误”，再把这份责任分摊给更前面的层，直到每个参数都分到自己的一份。

算一遍所有参数的梯度，计算量大约只是正向计算的两倍。PyTorch 等深度学习框架会自动完成这件事，写模型的人只需定义正向怎么算。[19] 有了梯度，具体怎么更新参数由**优化器**决定，实际常用的是 **Adam**：它参考最近几步梯度的平均方向（像滚下山的球有惯性），还会给每个参数单独调整步子大小。[20]

梯度消失：深网络为什么难训练

从上面的例子能看出，梯度每往回传一层，就要**乘上**这一层的局部系数。层数一多，就是一长串连乘：

```
每层系数 0.5，传 30 层：0.5³⁰ ≈ 0.000000001   → 梯度消失，前面的层几乎学不动
每层系数 1.5，传 30 层：1.5³⁰ ≈ 19 万          → 梯度爆炸，参数一更新就乱套
```

这就是深网络难训练的根本原因。前面提到的归一化和残差连接，正是为了解决这个问题。

归一化

同样的连乘，在数值往后传时也会出现。假设每过一层，数值的整体大小变成原来的 1.1 倍，看起来只多了 10%。可叠上 96 层之后：

```
每层放大 10%：1.1⁹⁶ ≈ 9400        （放大了近一万倍）
每层缩小 10%：0.9⁹⁶ ≈ 0.00004     （几乎缩没了）
```

数值太大，Softmax 会把几乎全部概率都给分数最高的那一个，计算还可能溢出；数值太小，有用的信息就淹没在误差里。而且每次调整参数后，前面各层输出的大小都会跟着变，后面的层得不停地适应，训练很不稳定。

归一化的做法很直接：**每层计算之前，先把向量的数值调回标准的大小范围。**

最常用的是**层归一化**（LayerNorm）：对每个位置的向量，先减去平均值，再除以标准差。[14] 以向量 (2, 4, 6, 8) 为例：

```
平均值 = 5
减去平均值 → (−3, −1, 1, 3)
标准差 = √[(9 + 1 + 1 + 9) / 4] = √5 ≈ 2.24
除以标准差 → (−1.34, −0.45, 0.45, 1.34)
```

如果输入变成 (200, 400, 600, 800)，归一化之后结果完全一样。**数值的整体大小被去掉了，各分量之间的相对关系保留了下来。** 这就像录音时的自动音量调节：不管说话人离麦克风多远，录下来的音量都差不多，说的内容却不变。

归一化之后，还会再乘上一组可学习的缩放系数（并加上一组偏移），让网络在需要时自己决定合适的大小，而不是被硬性固定。

每层数值的大小稳住了，梯度往回传时，每层的局部系数也就不至于离 1 太远。

现代大模型常用一种更简单的变体 **RMSNorm**：不减平均值，只除以“均方根”（各分量平方的平均值再开根号），效果差不多，计算更省。[15]

残差网络

有了归一化，深网络还是有一个怪现象。2015 年，何恺明等人发现：在 CIFAR-10 图像数据集上，56 层的普通网络，连**训练误差**都比 20 层的更高。[17]

这不是过拟合（过拟合是训练误差低、测试误差高），而是更深的网络连训练数据都学不好。这很奇怪：56 层的网络完全可以让前 20 层照搬 20 层网络的参数，后 36 层“什么都不做”，原样输出，效果至少不会更差。问题恰恰出在“什么都不做”上：一堆加权求和加弯折，要恰好学成“原样输出”，并不容易。

他们的解决办法是**残差连接**：**每一层不再输出一个全新的结果，而是算出对输入的修改，再加回到输入上。**

$$y = x + F(x)$$

这里 x 是这一层的输入，F 是这一层的计算。这样，“什么都不做”只需要让 F 输出 0，非常容易；需要修改时，F 也只用学“差多少”，而不必从头生成。

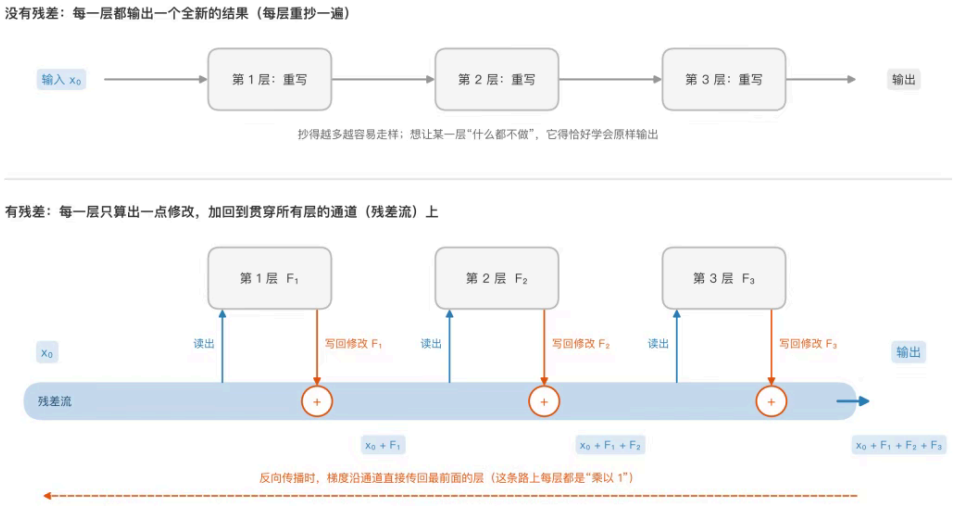
```
没有残差：向量 → 第 1 层重写 → 第 2 层重写 → ……（每层都重抄一遍）
有残差：  向量 → 加上第 1 层的修改 → 加上第 2 层的修改 → ……
```

好比在原稿上做批注，而不是每层都重抄一遍。抄得越多越容易走样；批注再多，原稿都还在。

把残差连接一层层展开：

$$x_L = x_0 + F_1 + F_2 + \cdots + F_L$$

其中 F_l 是第 l 层算出的修改。最终的结果，就是最初的输入加上每一层的修改。可以想象每个位置都有一条贯穿所有层的“信息通道”，叫**残差流**：每一层从通道里读出当前内容，算出一点修改，再写回通道。



残差连接对训练还有一个更关键的好处：**它给梯度修了一条“直通路”。** 残差层的输出是 $y = x + F(x)$，梯度往回传时，局部系数是“1 加上 F 的系数”：

```
没有残差：y = F(x),        局部系数 = F 的系数，比如 0.01
有残差：  y = x + F(x),    局部系数 = 1 + F 的系数，比如 1.01

传 30 层：0.01³⁰ = 10⁻⁶⁰（彻底消失）；1.01³⁰ ≈ 1.35（完好传回）
```

不管中间的层学成什么样，梯度总有一条“乘以 1”的路，能从损失一路传回最前面的层。 这就是残差网络能叠到上百层的关键。

用上残差连接后，他们训练出了 152 层的网络，拿下了 2015 年 ImageNet 图像识别比赛的冠军。今天的大语言模型，每一层都用了残差连接。

背答案还是学方法：过拟合与泛化

通用逼近定理没回答的最后一个问题是：**在训练数据上做得好，换成没见过的数据，还能做好吗？** 回答它，也就回答了熵一节留下的问题：为什么“压得短”意味着“学到了规律”。

假设给两个学生看一万道乘法题和答案：

```
23 × 17 → 391
46 × 12 → 552
81 × 35 → 2835
……
```

学生 A 把一万道题的答案全背下来，学生 B 学会了乘法的方法。然后出一道没见过的题：

```
67 × 24 → ?
```

A 答不出来，B 能算出来。我们会说：B 不只是记住了，而是学会了。

再换个角度问：**要把这一万道题的答案存下来，谁需要记的东西更少？** A 要存一万个答案；B 只需要记一套乘法规则，答案都能现算出来。

```
发现一条可复用的规律
      |
      ├── 不用逐条记住所有答案 → 压缩
      |
      └── 能处理没见过的新题   → 泛化
```

压缩和泛化来自同一个源头：不是先压缩、然后突然变聪明，而是找到了规律，就同时获得了压缩能力和解决新问题的能力。 把学习看作寻找数据最简洁的描述，正是**最小描述长度原则**的核心思想。[21]

神经网络也会面临这两条路。参数足够多时，它完全可以像学生 A 那样，把训练数据硬记下来。这时训练数据上的交叉熵很低，换成新文本就不行了，这叫**过拟合**。

所以训练时通常会留出一部分模型从没见过的文本，叫**验证集**，只用来检查、不参与训练。比较两条损失曲线：

训练集上的交叉熵	验证集上的交叉熵	说明
下降	同步下降	学到的东西能推广，训练正常
继续下降	不降反升	开始死记训练数据，过拟合了
都很高	都很高	模型太小或训练不够，没学会

大语言模型的预训练有个特点：数据量极大，大部分文本只被看一遍或几遍，模型根本来不及逐条背诵，硬记的办法走不通，只能去找可复用的规律。所以经典意义上的过拟合相对少见。但对训练数据里重复出现很多次的内容，模型仍可能逐字记住，这会带来隐私和版权问题。

判断模型学没学会，要看它在没见过的文本上的表现，而不是训练数据上的分数。

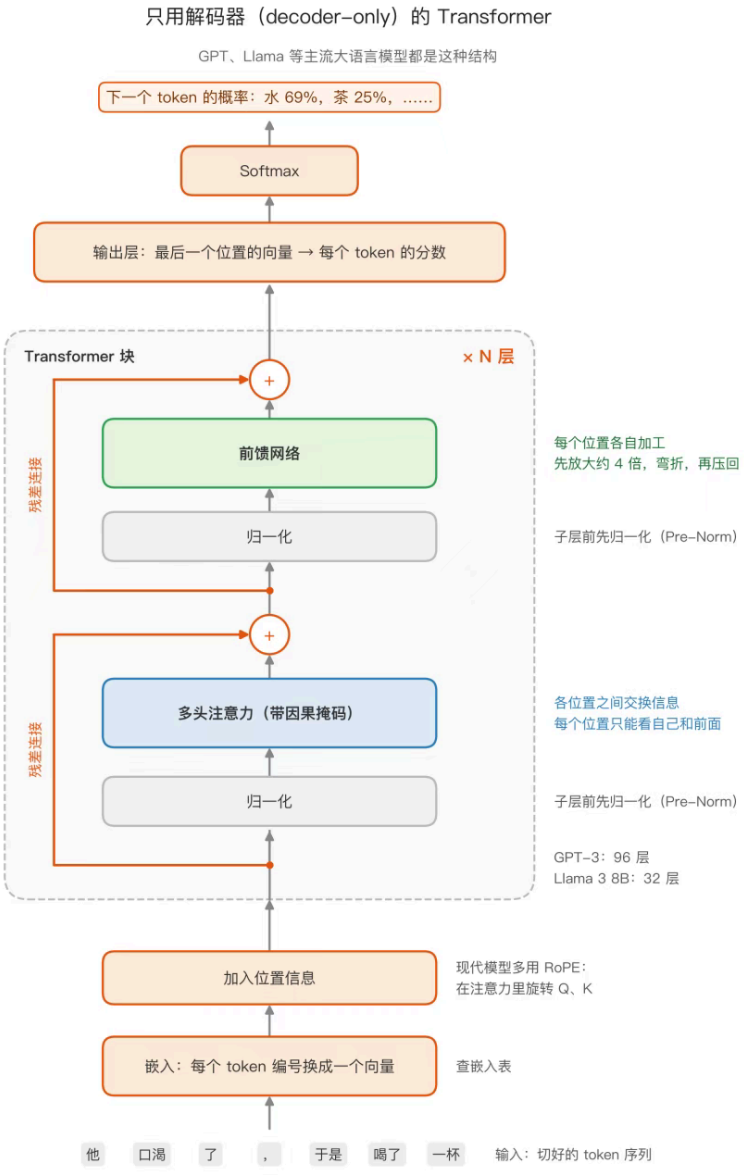
到这里，网络怎么搭、怎么学、怎样才算学会都有了。剩下的问题是：具体用什么结构，才能把一整段前文读进去，算出下一个 token 的概率？

Transformer

2017 年，Google 的论文《Attention Is All You Need》提出了 **Transformer** 架构。[25] 如今主流的大语言模型几乎都基于它。原始的 Transformer 是为翻译设计的，分成两半：**编码器**负责读懂原文，**解码器**负责一个词一个词地写出译文。语言模型只需要“接着往下写”，所以只用了解码器这一半。

先看整体结构，后面逐个解释：

```
token 编号序列
    ↓ 嵌入：每个编号换成一个向量
    ↓ 加入位置信息
    ↓ Transformer 块 × N 层，每块包含：
    |      注意力：各个位置之间交换信息
    |      前馈网络：每个位置各自加工
    |      （每个子层前做归一化，外面套着残差连接）
    ↓ 输出层：把最后一个位置的向量变成每个 token 的分数
    ↓ Softmax
下一个 token 的概率
```



归一化、残差连接、前馈网络（就是前面讲的“加权求和 + 弯折”）都已经讲过。下面顺着数据流动的方向，先讲文字怎么变成向量，再讲真正的核心：**注意力**。

嵌入：把编号变成有含义的向量

前面一直说“输入前文”，可神经网络只会做加减乘除，输入必须是数字。分词之后，每个 token 都有一个编号，比如“水”是 452，“茶”是 25。能不能直接把编号送进网络？

不行。**编号本身没有含义**：452 并不比 25“更大”，编号相邻的两个 token 意思也未必相近。直接用编号，网络会误以为 token 之间有大小关系。

用一串属性来描述

想让机器分清苹果和梨，只给它们编号是不够的，得把各种属性都拉出来，列成一张清单，每项打个分（示意）：

	甜	脆	红	圆	是水果	能喝
苹果	0.7	0.8	0.8	0.9	1	0
梨	0.8	0.6	0.1	0.6	1	0
水	0	0	0	0	0	1

对着清单，机器就能“看出”：苹果和梨大部分属性都接近，差别主要在红不红、圆不圆；它们和水几乎没有一项相同。**这样一串数字就是前面说过的向量，每一项属性是一个维度。**

嵌入就是这个思路：**给每个 token 分配一个向量**，大模型里通常有几千维。所有 token 的向量存在一张**嵌入表**里，查表就能把编号换成向量。以 Llama 3 的 80 亿参数版本为例，词表约 12.8 万个 token，每个向量 4096 维，嵌入表就有约 5.25 亿个数字。[11]

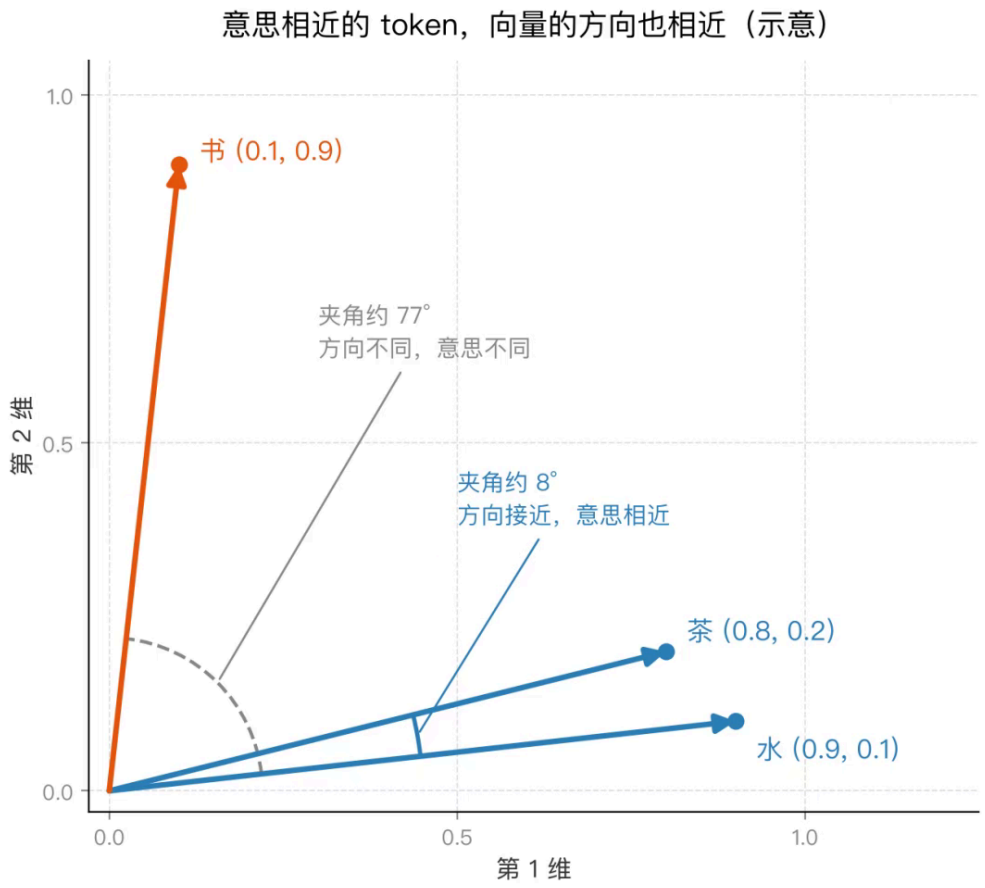
和上面的清单不同，**每一维代表什么，不是人定的。** 这些向量也是参数，和网络里的其他参数一样，一开始是随机的，靠反向传播一起训练出来。

训练会让意思相近的 token 靠在一起

用二维向量做个示意：

token	向量（示意）
水	(0.9, 0.1)
茶	(0.8, 0.2)
书	(0.1, 0.9)

把向量看成从原点出发的箭头，“水”和“茶”方向接近，“书”指向别处。



这些位置不是人安排的。“水”和“茶”经常出现在相似的上下文里，比如“喝了一杯_”“_凉了”，让它们的向量相近，对这些位置的预测都有帮助，梯度就会把它们推到一起。

这正好解决了神经网络一章开头的难题：没见过的前文，统计表里查不到。 模型在“口渴了喝一杯水”里学到的东西，会自然地迁移到向量相近的“茶”“饮料”上。“沙漠里走了三天三夜，终于喝到了一杯____”虽然从没见过，但“沙漠”“三天三夜”在别的文本里常和“口渴”“缺水”一起出现，它们的向量会带上相近的特征，模型就能把“喝水”的规律用过来。这正是 2003 年神经网络语言模型被提出时的核心想法。[22]

向量的常见操作

操作	怎么算	代表什么
点积	对应位置相乘，再加起来	两个东西有多像
加法	对应位置相加	把两组特征叠在一起
减法	对应位置相减	两者差在哪，也就是一种“关系”
数乘	每个位置乘同一个数	方向不变，特征整体变强或变弱

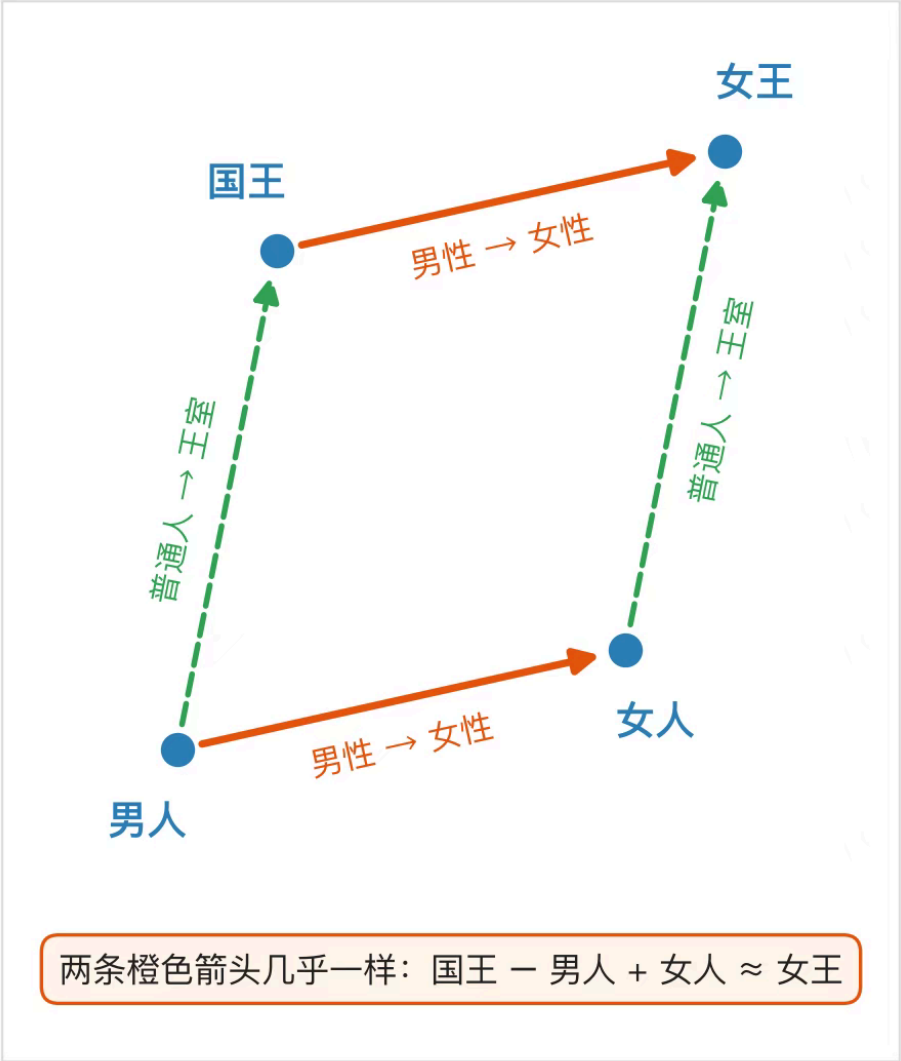
点积衡量方向有多接近：方向越接近，点积越大；两个方向垂直时，点积为 0，表示毫不相干。点积在后面的注意力里会反复出现。

```
水 · 茶 = 0.9 × 0.8 + 0.1 × 0.2 = 0.74    （方向接近，点积大）
水 · 书 = 0.9 × 0.1 + 0.1 × 0.9 = 0.18    （方向不同，点积小）
```

加法是把两组特征叠在一起。按前面的属性清单来理解：两个向量相加，每项属性的强度各自相加，结果同时带有两边的特征。这样加之所以有意义，是因为网络读取向量的方式就是点积（加权求和），而点积对加法可以拆开：$(a+b)\cdot w=a\cdot w+b\cdot w$。两个向量加在一起，后面的层读出的正好是两部分各自的贡献之和，谁也没抹掉谁。前面讲的残差流，就是靠这一点把每层的修改直接加回去；后面的注意力和位置信息，也都会用到加法。

减法取出的是两者的差别，也就是一种关系。word2vec 展示过："国王" − "男人" + "女人"，结果最接近"女王"。[23] 换个写法就是"女王" − "国王" ≈ "女人" − "男人"：从男性到女性，是空间里一个大致固定的方向。

词与词的关系，是空间里一个大致固定的方向（示意）



向量的大小重要吗？ 含义主要在方向上，大小更像"音量"。把"茶"乘以 2 变成 (1.6, 0.4)，它和"水"的点积从 0.74 变成 1.48，意思却没变。所以比较两个词像不像，通常只看夹角，把点积除以两个向量的长度，这叫**余弦相似度**："水"和"茶"约 0.99，"水"和"书"约 0.22，向量放大多少倍都不变。不过在网络内部，大小会跟着点积一起放大，影响 Softmax 分配得有多集中，而且容易失控，这正是每层都要先做归一化的原因。

几千维够用吗？ 语言里的概念远不止几千个。如果每个概念都要占一个和其他概念严格垂直的方向，4096 维最多只能放 4096 个。但只要允许一点误差，比如夹角在 88°～92° 之间，情况就完全不同：高维空间里随便取两个方向，几乎总是接近垂直。三维空间里，随机

两个方向的夹角落在这个范围的概率只有约 3.5%；在 GPT-3 的 12288 维里，超过 99.98%。

这种“几乎垂直”的方向能放下的数量随维度指数增长，远远超过维度本身。于是模型可以把远多于维度数的概念，各自放在一个几乎互不干扰的方向上，这叫**叠加**（superposition）。[24] 代价是读一个概念时，会混进一点其他概念的噪声；好在一段话里同时用到的概念很少，这点噪声通常可以容忍。

两点提醒：

- 真实向量的某一维，通常不能单独解释成“甜”“是不是水果”这类含义。概念对应的是方向，又彼此叠加，往往分散在许多维度的组合里。
- **嵌入表给每个 token 的是一个固定的向量，不管上下文是什么。**
  “苹果”在“吃苹果”和“苹果手机”里拿到的是同一个向量。

让向量随上下文改变，需要每个位置都能看到前文的其他位置。这正是注意力要做的事。

注意力

先看两句话：

```
他口渴了，于是喝了一杯____
他困了，于是喝了一杯____
```

第一句大概率填“水”，第二句更可能是“茶”或“咖啡”。可在“一杯”这个位置上，决定答案的“口渴”和“困”在好几个 token 之前。再看：

```
奖杯放不进箱子，因为它太大了。   → “它”指奖杯
奖杯放不进箱子，因为它太小了。   → “它”指箱子
```

只改一个字，“它”指的对象就变了。

所以，**每个位置都必须能从前文的其他位置收集信息。** 最简单的办法是把前面所有位置的向量平均一下，但这样有用的信息会被大量无关的词冲淡：“口渴”和“了”“，”“于是”的分量一样重。

真正需要的是**有选择地收集**：对当前位置有用的，多收一点；无关的，少收或不收。而且“什么有用”要随当前位置和上下文而变。这就是**注意力机制**：

```
1. 给前文的每个位置打一个“相关程度”分数
2. 用 Softmax 把分数变成权重（都是正数，加起来等于 1）
3. 按权重对各位置的信息加权平均
```

注意力的结果会加回到当前位置的向量上（残差连接）。经过注意力之后，“一杯”这个位置的向量里就融进了“口渴”的信息；“苹果”的向量也会因为旁边是“吃”还是“手机”而变得不同。

只能往前看：因果掩码

生成时，模型只知道已经写出的内容，训练时也必须保持一致：每个位置只能看它自己和前面的位置，否则就是偷看答案。

当前位置 \ 能看到	他	口渴	喝了	一杯
他	✓			
口渴	✓	✓		
喝了	✓	✓	✓	
一杯	✓	✓	✓	✓

做法是把“未来位置”的分数设为负无穷，经过 Softmax 后权重就是 0。这叫**因果掩码**。

它还带来一个好处：一段文本只需算一次，每个位置都同时在预测自己的下一个 token，互不偷看。一段 1000 个 token 的文本，一次就能提供约 1000 道练习题，这是 Transformer 训练效率高的重要原因。

“相关程度”的分数具体怎么算？这就要用到查询、键和值。

查询、键、值（QKV）

可以把注意力想象成每个位置都在做一次小型检索。每个位置根据自己的向量，算出三样东西：

名称	直观含义	类比：在图书馆找书
查询 Q（Query）	我在找什么信息？	你想查的主题
键 K（Key）	我这里有什么信息？	每本书书脊上的标签
值 V（Value）	如果你关注我，我能提供的内容	书的正文

计算过程：

```
1. 当前位置用自己的查询，和每个可见位置的键做点积，得到分数
2. Softmax，把分数变成权重
3. 按权重对各位置的值加权平均
```

Q、K、V 都是用 token 的向量，分别乘以三个参数矩阵 W_Q、W_K、W_V 算出来的。**矩阵**就是一张排成行和列的数字表；向量乘以矩阵，相当于一层神经元同时做很多组加权求和（不做弯折），得到一个新的向量。**这三个矩阵是训练出来的**：训练决定了每个位置该“找什么”、该“贴什么标签”、该“提供什么内容”。

手算一次

前文是“他 口渴 喝了 一杯”，现在要为最后一个位置“一杯”收集信息，以便预测下一个token。

为了便于理解，假设向量只有两维，第 1 维大致表示“和饮水有关”，第 2 维大致表示“指人物”（真实模型的维度不能这样简单解释）。各位置的键和值如下（教学构造）：

位置	键	值
他	(0, 2)	(0, 1)
口渴	(2, 0)	(1, 0)
喝了	(1.5, 0)	(0.8, 0)
一杯	(0.5, 0)	(0.3, 0)

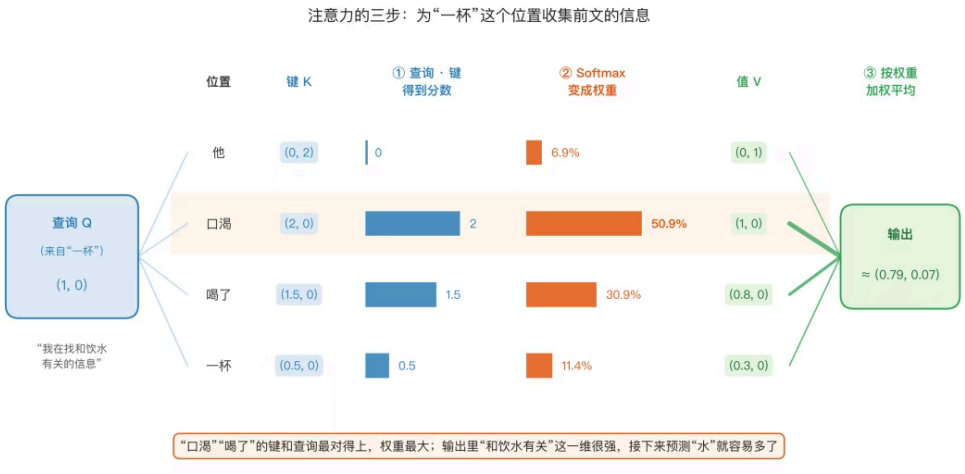
“一杯”的查询是 (1, 0)，意思大致是“我在找和饮水有关的信息”。

第 1 步：查询和每个键做点积，得到分数。第 2 步：Softmax，把分数变成权重。

位置	点积	分数	权重
他	1×0 + 0×2	0	6.9%
口渴	1×2 + 0×0	2	**50.9%**
喝了	1×1.5 + 0×0	1.5	30.9%
一杯	1×0.5 + 0×0	0.5	11.4%

第 3 步：按权重对值加权平均。

```
第 1 维 = 6.9%×0 + 50.9%×1 + 30.9%×0.8 + 11.4%×0.3 ≈ 0.79
第 2 维 = 6.9%×1 + 50.9%×0 + 30.9%×0   + 11.4%×0   ≈ 0.07
输出 ≈ (0.79, 0.07)
```



“一杯”收集到的信息主要来自“口渴”和“喝了”，“和饮水有关”这一维很强。带着这些信息，预测“水”就容易多了。

没有人规定“应该关注口渴”。 是训练中这样关注有助于降低交叉熵，相应的 W_Q、W_K 才逐渐形成。真实模型的注意力权重，通常也没有这么整齐易读。

一个细节：缩放。 实际计算时，分数要先除以向量维度的平方根 $\sqrt{(d)}$。维度越高，点积是越多项相加，数值越大；分数差距太大时，Softmax 会把几乎全部权重给一个位置，梯度也会变得非常小。[25] 在上面的例子里，除以 $\sqrt{(2)}$ 之后，“口渴”的权重从 50.9% 变成约 43.6%，分布更平缓一些。

把所有位置写成矩阵，整个注意力就是一行公式：

$$Attention(Q,K,V)=\text{softmax}((QK^T)/(\sqrt{(d)}))V$$

QK^T 是所有查询和所有键两两做点积，除以 $\sqrt{(d)}$ 是缩放，softmax 把分数变成权重，最后乘以 V 就是按权重对值加权平均。

多头注意力

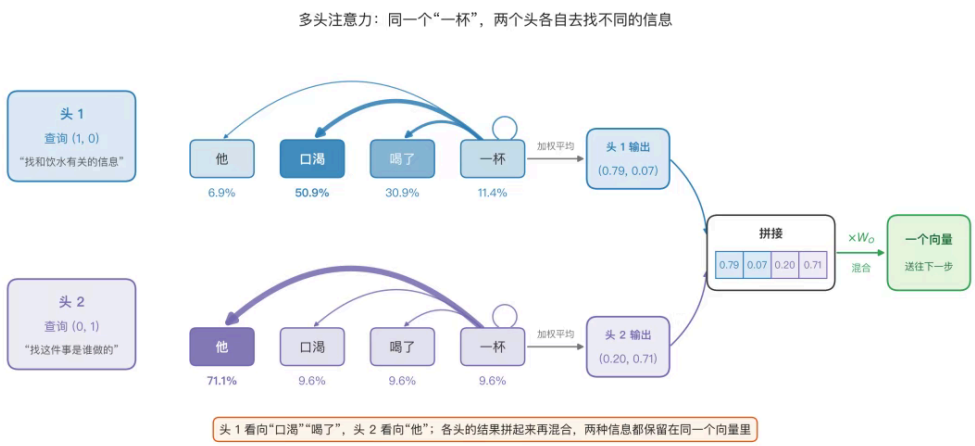
一句话里同时存在多种关系：谁做了什么、词语搭配、指代、话题……一组 Q、K、V 只能按一种“查询”去找。

还是上面的例子，如果“一杯”的查询换成 (0, 1)，意思大致是“这件事是谁做的”：

位置	分数	权重
他	2	**71.1%**
口渴	0	9.6%
喝了	0	9.6%
一杯	0	9.6%

这次注意力集中到了“他”身上。

所以 Transformer 会**同时使用多组独立的 Q、K、V 矩阵**，每组叫一个**注意力头**。每个头各自算一遍注意力，最后把所有头的结果拼接起来，再用一个矩阵混合成一个向量。



研究者在真实模型里找到过一些功能清晰的头。例如**归纳头**：前文出现过“A B”，之后再遇到“A”时，它会去关注上次“A”后面的“B”，从而预测下一个是“B”。比如前文出现过“哈利·波特”，后面再写到“哈利·”，它就能把“波特”找出来。有研究认为，这类头是模型能从提示里“现学”规律的重要机制之一。[26]

位置信息

注意力只看“相关程度”，本身并不知道顺序。在上面的计算里，把“他”和“口渴”交换位置，权重结果完全不变。可“狗咬人”和“人咬狗”用字完全相同，意思却截然相反。所以必须额外加入位置信息。

原始 Transformer 的做法： 给每个位置生成一个特定的向量（用不同频率的正弦、余弦函数算出来），加到 token 的向量上。这样同一个“狗”出现在第 1 位和第 3 位时，向量就不一样了。

现代模型的常见做法：旋转位置编码（RoPE）。 它不改 token 向量本身，而是在算注意力时，把查询和键按“和位置成正比的角度”旋转一下。[27]

用二维向量看最直观。假设每往后一个位置，就多转 10°：

```
第 3 个位置的查询，转 30°
第 5 个位置的键，  转 50°
```

两个向量的点积只取决于它们的长度和夹角。旋转后，两者之间多出来的夹角是 50° − 30° = 20°，只和**两个位置相差 2** 有关。换成第 103 和第 105 个位置，各自转 1030° 和 1050°，多出来的夹角还是 20°，点积结果完全一样。

所以 RoPE 让注意力分数只取决于**相对距离**，而不是绝对位置。这通常更有用：紧挨着“一杯”的词，往往比 500 个 token 之前的词更相关，不管这句话出现在文章的哪一行。真实的 RoPE 会把向量分成很多对二维分量，每一对用不同的旋转速度：转得快的对近距离敏感，转得慢的能感知远距离。Llama 等许多模型都使用了 RoPE。

前馈网络

注意力完成了“位置之间交换信息”，接下来每个位置还要各自经过一个**前馈网络**，也就是神经网络一章讲的“加权求和 + 弯折”。

它的结构很简单：先把向量放大到约 4 倍宽，经过非线性函数，再压回原来的宽度。以 GPT-3 为例，向量宽度是 12288，前馈网络中间层宽度是 49152。[10] 现代模型常把 ReLU 换成带“门控”的变体（如 SwiGLU），但思路不变。[28]

组件	作用	类比
注意力	从其他位置收集信息	开会交流
前馈网络	加工收集到的信息	回到工位各自消化

注意力主要是在做加权平均，真正“弯折”、做复杂变换的，主要是前馈网络。通用逼近定理说的那种表达能力，很大程度上就来自这里。

前馈网络约占 Transformer 参数的三分之二。有研究发现，它的行为类似“键-值记忆”：一部分参数负责识别输入中的某种模式，另一部分据此提高某些后续 token 的概率。比如识别出“法国的首都是”这个模式，就提高“巴黎”的分数。[13] 但知识分散在大量参数中，不能指着某个参数说“这里存着‘巴黎是法国首都’”。

把所有部件拼起来

一个 Transformer 块由两个**子层**组成：注意力和前馈网络。每个子层前做归一化，外面套着残差连接：

```
x = x + 注意力(归一化(x))      ← 各位置交换信息，写回残差流
x = x + 前馈网络(归一化(x))    ← 各位置各自加工，写回残差流
```

归一化放在子层的**前面**（叫 Pre-Norm），而不是像最初的 Transformer 那样放在残差相加之后，训练更稳定，如今的大模型基本都这么做。[16]

把这样的块堆叠几十层（GPT-3 有 96 层），完整走一遍：

```
输入：“他口渴了，于是喝了一杯”
    ↓ 分词：他 | 口渴 | 了 | ， | 于是 | 喝了 | 一杯
    ↓ 嵌入：每个 token 换成一个几千维的向量，放进各自的残差流
    ↓ 第 1 层：注意力（各位置交换信息）→ 前馈网络（各自加工）
    ↓ 第 2 层：在上一层的结果上继续加工
    ↓ ……共几十层
    ↓ 取最后一个位置的向量，算出每个 token 的分数
    ↓ Softmax
输出：水 69%，茶 25%，……（示意）
```

越往后的层，每个位置的向量融合的上下文就越多。到最后一层，“一杯”这个位置的向量，已经在很大程度上代表了“读完前面整句话之后，接下来该说什么”。

参数都在哪里？ 设向量宽度为 d，一个 Transformer 块的主要参数是：

部分	矩阵	参数量
注意力	Q、K、V、输出，4 个 d × d 矩阵	$4d^2$
前馈网络	d → 4d、4d → d，2 个矩阵	$8d^2$
合计		**$12d^2$**

代入 GPT-3 最大版本的数字，d = 12288，96 层，词表约 5 万：

```
每层：   12 × 12288² ≈ 18.1 亿
96 层：  18.1 亿 × 96 ≈ 1739 亿
```

```
嵌入表：50257 × 12288 ≈ 6.2 亿
合计 ≈ 1746 亿，与公布的 1750 亿基本一致
```

归一化层、偏置等参数很少，可以忽略。**参数的大头在每层的矩阵里，其中前馈网络约占三分之二。**

到这里，模型本身就讲完了：给它一段前文，它能算出下一个 token 的概率表。可我们在聊天框里看到的，是一个能一问一答的助手。从一张概率表到 ChatGPT，中间还隔着好几步。

从模型到 ChatGPT

这一章依次回答五个问题：一张概率表怎么变成一段话？只会续写的模型，怎么变成聊天助手？它怎么联网、看图、用工具？模型为什么越做越大？它又有哪些做不好的地方？

自回归生成

开头说过，生成就是一个循环：从概率表里选出一个 token，接到前文后面，再交给模型预测下一个。自己的输出成为下一步的输入，这种方式叫**自回归生成**。

什么时候停？模型选出了表示“结束”的特殊 token，或者达到了预设的最大长度。

这个循环有一个重要的推论：**已经写出的 token 不会撤回。** 如果前面选了一个不太好的词，后面只能顺着它往下写。例如模型一开头就写了“答案是 A”，即使后面的推理发现 A 不对，它也常常会想办法为 A 圆场，而不是推翻重来。

采样：从概率表里挑一个

还是“他口渴了，于是喝了一杯____”，模型给出水 69%、茶 25%、书 6%。挑法有好几种：

策略	做法	特点
贪心	永远选概率最高的	稳定，但容易单调、重复
随机采样	按概率抽签	更多样，但偶尔会抽到离谱的低概率 token
Top-k	只在概率最高的 k 个里抽	截掉长尾，但 k 是固定的
Top-p（核采样）	从高到低累加概率，达到 p 就停，只在这些里抽	候选数量随情况自动变化

随机采样的风险在长尾。 词表有十几万个 token，单个离谱 token 的概率也许只有十万分之一，可成千上万个加起来，就可能占到几个百分点。生成几百个 token，碰上几次离谱选择几乎是必然的，而一旦选错，后面只能顺着写。

Top-p 以 p = 90% 为例：水 69% + 茶 25% = 94%，已经超过 90%，所以只在“水”和“茶”里抽，按比例重新分配后约为 73% 和 27%，“书”被排除。[29]

Top-p 比 Top-k 灵活的地方在于，候选数量会随情况变化：

- 前文是“床前明月____”时，“光”一个就占了 99%，Top-p 只留下它；Top-k 却仍然硬留 k 个，可能把不该有的候选也放进来。
- 前文是“我今天想吃____”时，合理的候选有几百个，每个都不高，Top-p 会留下很多；Top-k 却可能只留几个，把合理的也砍掉了。

那为什么不干脆每次都选最高的？研究发现，在开放式写作和对话里，一味追求“最可能”的文本往往平淡、重复，甚至陷入同一句话反复出现的循环；而人写的文字，恰恰包含不少“不那么可能”的词。[29] 这和信息量一节说的一致：**完全猜得到的文字，没有信息。**

温度：保守还是大胆

温度是另一个常用的旋钮：先把每个分数除以温度，再做 Softmax。

温度	水	茶	书	效果
0.5	约 88%	约 12%	约 0.6%	更集中、更保守
1	约 69%	约 25%	约 6%	原始分布
2	约 53%	约 32%	约 15%	更平均、更多样，也更容易出错

温度 0.5 相当于把分数翻倍，原本“差 1 分”变成“差 2 分”，Softmax 之后差距被放大；温度 2 相当于把分数减半，差距被缩小。温度趋近 0 时，最高分拿走几乎全部概率，相当于贪心；温度很高时，分数差距几乎被抹平，接近在所有候选里随便抽。

任务	通常的设置
写代码、回答事实问题、提取信息	低温度，结果稳定
日常对话、写作	中等温度
头脑风暴、写诗、起名字	较高温度，多样性更重要

这也解释了**为什么同一个问题，每次回答不一样**：同样的前文得到同样的概率表，但每次抽出的 token 可能不同；一旦某一步不同，后面整段都会走向不同的方向。

最后要注意：**采样只能在模型给出的概率里挑，改变不了模型本身的理解能力。** 概率表本身错了，怎么挑都救不回来。

基座模型：只会续写

到目前为止，模型学的都是“人写的文本接下来通常是什么”。这一步叫**预训练**，得到的是**基座模型**。它只会续写，不会“回答”：

```
输入：法国的首都是哪里？
基座模型可能的续写（示意）：德国的首都是哪里？英国的首都是哪里？请在括号内填写答案……
```

因为网上类似的文字常常出现在习题列表里。它其实“知道”答案，只是不知道自己应该扮演回答问题的人。把问题伪装成“一段通常后面跟着答案的文本”，它就能答对：

```
问：日本的首都是哪里？
答：东京。
问：法国的首都是哪里？
答：____                          → 巴黎
```

这说明**知识在预训练里已经学到了，缺的是“以助手身份回应”的行为方式。** 基座模型还可能一直写下去不知道停，模仿网上的粗鲁语气，或者对有害请求照单全收。把它变成助手的这一系列训练，统称**后训练**。

监督微调：看示范学回答

监督微调（SFT） 的做法是：收集大量“指令 + 优质回答”的示范，用同样的“预测下一个 token”方式继续训练。对话会被整理成带特殊标记的格式（示意，不同模型的标记各不相同）：

```
<|系统|>你是一个乐于助人的助手。
<|用户|>法国的首都是哪里？
<|助手|>法国的首都是巴黎。<|结束|>
```

训练时通常只对“助手”部分计算交叉熵：模型要学的是怎样回答，而不是怎样提问。

对模型来说，对话仍然是一段连续的文本，“对话”只是用特殊标记组织起来的格式。 所谓“系统提示词”，就是放在最前面的一段文字；多轮对话中，之前每一轮的问答都会拼在一起，作为下一轮的前文；模型学会了在回答完时生成“结束”标记，所以知道什么时候停。

数据质量比数量更重要。 InstructGPT 的监督微调只用了约 1.3 万条提示；[30] LIMA 的研究只用 1000 条精心挑选的示范，就让一个 650 亿参数的基座模型有了相当不错的对话能力。[31]

RLHF：从人类偏好中学习

示范有两个局限：写高质量示范成本很高；而且很多时候，**人更擅长比较，而不是创作**，你可能写不出一首好诗，但给你两首，你大概能分辨哪首更好。

基于人类反馈的强化学习（RLHF） 就利用了这一点。**强化学习**是和“看示范”不同的另一种训练方式：不给标准答案，只给打分，让模型自己尝试，往得分高的方向调整。RLHF 分三步：[30]

```
1. 收集比较数据：对同一个问题让模型生成几个回答，请标注员选出更好的
2. 训练奖励模型：用这些比较数据，训练另一个模型给回答打分
3. 强化学习：模型生成回答 → 奖励模型打分 → 让高分回答更可能被生成，低分的更不可能
```

这里有一个关键约束：**不能让模型偏离原来太远。** 奖励模型只是人类偏好的近似，有盲点。放开优化的话，模型会找到钻空子的办法，比如奖励模型略微偏爱长回答，模型就越写越长、越写越空。这叫**奖励投机**。

效果如何？InstructGPT 的研究中，标注员更喜欢 13 亿参数的 InstructGPT 的回答，而不是 1750 亿参数的 GPT-3 的回答。[30] **就“好不好用”而言，后训练可能和模型规模一样重要。**

后来还出现了更简单的 **DPO（直接偏好优化）**：跳过奖励模型，直接用“较好 / 较差”的回答对去调整模型，提高较好回答的概率、降低较差回答的概率。[32]

人类偏好只能告诉模型“哪个回答更好”。可有些问题本来就有标准答案，能不能直接拿“对不对”来训练？

推理强化学习：只告诉它对不对

数学题、编程题的答案可以自动检查。于是可以让模型自己反复解题，答对就给奖励，答错就不给，不再需要人来比较。DeepSeek-R1 的报告显示，直接在基座模型上用这种方式训练（他们称这个版本为 R1-Zero），模型在 AIME 2024 数学竞赛题上的正确率从 15.6% 提升到 71.0%；而且训练中，模型自发地学会了先写出推理过程、中途检查、发现问题再回头修正。[33] **没有人教它“怎么想”，只告诉它“结果对不对”，它却自己摸索出了“多想一会儿”的策略。**

把从预训练到聊天助手的几个阶段放在一起看：

阶段	数据	得到什么
预训练	数万亿 token 的普通文本	基座模型：知识多，但只会续写
监督微调	数千到数十万条示范	能按指令回答
偏好对齐（RLHF、DPO）	人类或 AI 的比较数据	更有用、更得体、更安全
推理强化学习	可自动判对错的题目	更强的推理能力

联网、看图、用工具

现在的聊天助手还能联网搜索、看懂图片、运行代码。可模型只会预测下一个 token，这些是怎么做到的？

工具调用：模型只负责写出“要做什么”。 训练时，让模型学会在需要的时候，输出一段特定格式的文字（示意）：

```
<|用户|>今天北京天气怎么样?
<|助手|><|调用工具|>搜索：北京 今天 天气<|结束|>
```

模型写完这一行就停下，它自己并不会联网。外部程序看到这个格式，就去真正执行搜索，再把结果作为一段新文字拼到上下文后面，交还给模型：

```
<|工具结果|>北京今天晴，最高气温 22℃……
<|助手|>北京今天是晴天，最高 22℃，适合出门。<|结束|>
```

模型读到搜索结果，接着往下预测，就写出了回答。[34] 算数、运行代码、查资料、读文件，都是同一个套路：**模型写出要做什么，外部程序去做，结果再变回文字交给模型。** 平时说的 **Agent（智能体）**，就是让这个循环自动跑很多轮：想好下一步 → 调用工具 → 看结果 → 再决定下一步，直到任务完成。

看图：把图片也变成 token。 把图片切成许多小方块，再用一个专门的网络把每一块转成一个向量，和文字 token 的向量排在一起，送进同一个 Transformer。[35] 对模型来说，一张图就是一串特殊的 token。

所以，不管是聊天、用工具还是看图，**对模型来说，一切都是一串 token**。它做的始终只有一件事：根据前文，预测下一个。

规模定律

标题叫“大语言模型”，到底“大”在哪里，为什么要大？先看影响模型能力的三个核心量：

量	含义	类比
参数量 N	模型里可调的数字有多少	脑容量
训练 token 数 D	训练时读过多少文本	读过多少书
计算量 C	训练一共做了多少次运算	花了多少学习时间

三者之间有一个常用的估算：**训练计算量 ≈ 6 × N × D**。每个 token 经过模型一次，每个参数大约参与 1 次乘法和 1 次加法，正向约 2N 次运算；反向传播约是正向的两倍，约 4N 次，合起来约 6N。[36] 代入 GPT-3：6 × 1750 亿 × 3000 亿 ≈ 3.15×10^{23}，和论文报告的 3.14×10^{23} 几乎一样。[10]

2020 年的一项研究系统地测量了交叉熵和规模的关系，发现在跨越多个数量级的范围内，交叉熵随参数量、数据量、计算量的增加，按**幂律**平稳下降：**规模每扩大相同的倍数，交叉熵就下降一个大致固定的比例。**[36]

只扩大这一项	扩大 10 倍后，交叉熵约变为原来的
参数量	84%
数据量	80%
计算量（最优分配时）	89%

画在横纵轴都取对数的图上，这几乎是一条直线。这带来一个很实用的结果：**先用小模型做一批便宜的实验，把直线延长，就能预测大模型大约能达到多低的交叉熵**，再决定要投入多少资源。

但也要注意：每扩大 10 倍只降一点，所以想明显降低交叉熵，规模往往要扩大好几个数量级。这就是模型越做越大的原因。

参数和数据要一起涨。 2022 年的 Chinchilla 研究发现，此前很多大模型“参数多、数据少”；在固定的计算预算下，每个参数大约对应 20 个训练 token 比较划算。他们用和 2800 亿参数的 Gopher 相当的算力，训练了一个只有 700 亿参数、但读了 1.4 万亿 token 的模型，结果在许多任务上都超过了 Gopher。[37] 后来很多模型还会用远超这个比例的数据去训练较小的模型，比如 Llama 3 的 80 亿参数版本读了约 15 万亿 token，[11] 因为模型只训练一次，却要被使用无数次，而使用成本取决于参数量。

损失降不到零

交叉熵会一直降到零吗？不会。Chinchilla 研究把交叉熵拆成三部分来拟合：[37]

```
交叉熵 ≈ 不可约的部分 + 模型不够大带来的部分 + 数据不够多带来的部分
```

模型越大，第二项越小；数据越多，第三项越小。但**第一项无论如何都消除不了**，他们拟合出的值约为每个 token 1.69 nat（约 2.4 bit）。论文的解释是：它对应的是自然文本本身的熵。

这和前面完全接上了：**交叉熵永远不低于熵。** 文本里有真正的随机性，比如作者下一句想写什么、一个人叫什么名字、明天的新闻是什么，再聪明的模型也不可能百分之百猜中。规模定律描述的，正是模型的交叉熵如何一步步逼近语言本身的熵，也就是香农当年想测量的那个数。

（这个数值按 token 计算，依赖于具体的数据集和分词方法，不能和香农按字母估算的 0.6～1.3 bit 直接比较。）

规模能把交叉熵一步步压低，但模型的能力终究来自“在海量文本上预测下一个 token”，它的局限也大多来自这里。

幻觉

幻觉指模型生成流畅、自信，但与事实不符的内容，比如编造不存在的论文、网址、引语，或者给出一个精确但错误的数字。[38] 用前面讲过的东西，可以从四个角度理解：

1. 训练目标是“像”，不是“真”。 交叉熵衡量的是“像不像人写的文本”。遇到不确定的内容时，最“像样”的续写，往往仍是一段流畅、自信的文字：一篇论文的引用格式应该长什么样，模型非常清楚；但这篇论文是否存在，它未必知道。

2. 零散的事实没有规律可循。 模型靠找到可复用的规律来压缩数据，但“某个普通人的生日是哪天”这类事实没有规律，只能逐条记住，而只在训练数据里出现一两次的事实很难记牢。有研究指出：如果训练数据中 20% 的生日信息只出现过一次，那么可以预期，基座模型在这类问题上至少有约 20% 会答错。[39]

3. 评测奖励“猜”。 很多评测只看答案对不对，回答“不知道”和答错一样得零分。就像考试不倒扣分时，蒙一个总比空着强。在这种激励下优化出来的模型，自然倾向于猜一个答案，而不是承认不确定。[39]

4. 写出的字收不回来。 模型一旦开头写了“这篇论文发表于 2019 年的……”，后面只能顺着编下去。

越是冷门、越是具体（人名、数字、日期、引用、网址），越容易出错。 应对办法：提供资料让它依据资料作答；明确允许它说“不知道”；要求注明出处并自己核实；关键信息一定多方验证。

其他常见局限

局限	根源	应对
知识有截止日期	只知道训练数据收集之前的内容，而且可能不知道自己不知道	涉及“最新”“现在”的问题，让它联网检索或自己核对
上下文有限	一次能看的 token 数有上限，注意力的计算量随长度的平方增长；窗口之内，放在中间的信息也容易被忽略[40]	重要要求放在开头或结尾，长对话适时总结
数字母、精确计算出错	看到的是 token 而不是一个个字符，比如 strawberry 可能被切成几块	让它写代码来算，或调用计算工具
迎合用户	人类评价时更容易给“同意我”的回答打高分，RLHF 把这种偏好也学了进去[41]	中立地提问，直接要求它指出问题
学到关联，不等于掌握因果	学的是文本里的方向性关联：学到“A 的母亲是 B”，不一定能答出“B 的儿子是谁”[42]	换个角度追问，关键推理自己检查

最后，模型的能力往往参差不齐。 它可能解得出一道复杂的数学竞赛题，却在一个简单的常识问题上犯错。**判断它可不可靠，最终要看它在没见过的、你自己的真实任务上表现如何。**

总结

回头看，整篇文章其实只回答了一串问题：

```
语言模型做什么?      给定前文，输出下一个 token 的概率
怎么衡量做得好不好?  交叉熵：越低，越接近人写作时的真实概率，压缩得也越短
预测的极限在哪里?    语言本身的熵：交叉熵永远不低于它
用什么来做预测?      神经网络：参数加非线性，能逼近任意函数；深度、归一化、残差让它能叠得很深
怎么把参数调好?      梯度下降 + 反向传播：在海量文本上一步步降低交叉熵
怎样才算学会?        在没见过的文本上也预测得好：找到可复用的规律，压缩和泛化同时发生
具体用什么结构?      Transformer：注意力收集上下文，前馈网络加工信息，几十层堆叠
一段话怎么写出来?    自回归生成：从概率里挑一个，接上，再预测；温度和 Top-p 决定怎么挑
怎么变成聊天助手?    后训练：监督微调学格式，RLHF 学偏好，可验证奖励练推理
怎么联网、用工具?    写出调用工具的文字，外部程序去执行，结果拼回上下文；图片也变成 token
为什么要大?          规模定律：交叉熵随规模平稳下降，一步步逼近语言的熵
它做不好什么?        幻觉、知识截止、上下文有限……大多能从“预测下一个 token”里找到根源
```

用一首诗，再把全文串一遍：

> 万卷人言化作丝，层层网络织新辞。
> 前文照应千般意，概率权衡一句诗。

> 梯度回传修旧失，人间偏好亦为师。
> 纵然出口皆成锦，几处真知几处疑。

留给读者的问题

上面这串问题都有了答案。最后留几个还没有定论的问题，供读者思考：

- **现在的架构是最优的吗？**
  人脑里没有反向传播，没有 Q、K、V，思考时也不是一个 token 一个 token 地往外蹦；一个孩子到十几岁接触的词不到 1 亿个，[43] 大模型却要读十几万亿个 token。[11] 这说明现在的做法还有很大的改进空间，还是飞机本来就不必像鸟一样扇翅膀？
- **不靠人类的语言数据，能达到同样的智力吗？**
  AlphaGo Zero 不看人类棋谱，只靠自己和自己下，就超过了所有人类棋手。[44] 可围棋有明确的规则和输赢，语言和现实世界里，谁来当裁判？
- **大模型有自我意识吗？**
  如果有，它在哪里：在训练好就不再改变的参数里，在每一场对话里，还是在每生成一个 token 的那一次计算里？一次对话结束，算不算一个意识的消亡？

参考资料

1. A. M. Turing, *Computing Machinery and Intelligence*, Mind, 59(236), 433–460, 1950。模仿游戏（图灵测试）。`https://doi.org/10.1093/mind/LIX.236.433`
2. Claude E. Shannon, *A Mathematical Theory of Communication*, 1948。信息量、熵与通信编码。`https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf`
3. MIT 18.310, *Shannon's Noiseless Coding Theorem*。熵是无损编码平均长度的下界。`https://math.mit.edu/~goemans/18310S15/noiseless-coding.pdf`
4. Hutter Prize 官方 FAQ。用无损压缩研究智能的动机。`http://prize.hutter1.net/hfaq.htm`
5. Claude E. Shannon, *Prediction and Entropy of Printed English*, Bell System Technical Journal, 30(1), 50–64, 1951。不同上下文长度下的英语熵、逐字预测实验与猜测序号编码。`https://www.princeton.edu/~wbialek/rome/refs/shannon_51.pdf`
6. Warren S. McCulloch, Walter Pitts, *A Logical Calculus of the Ideas Immanent in Nervous Activity*, Bulletin of Mathematical Biophysics, 5, 115–133, 1943。最早的人工神经元数学模型。`https://doi.org/10.1007/BF02478259`
7. George Cybenko, *Approximation by Superpositions of a Sigmoidal Function*, Mathematics of Control, Signals and Systems, 2, 303–314, 1989。单隐层网络的通用逼近。`https://doi.org/10.1007/BF02551274`
8. Kurt Hornik, Maxwell Stinchcombe, Halbert White, *Multilayer Feedforward Networks are Universal Approximators*, Neural Networks, 2(5), 359–366, 1989。`https://doi.org/10.1016/0893-6080(89)90020-8`
9. Moshe Leshno, Vladimir Ya. Lin, Allan Pinkus, Shimon Schocken, *Multilayer Feedforward Networks with a Nonpolynomial Activation Function Can Approximate Any Function*, Neural Networks, 6(6), 861–867, 1993。通用逼近对 ReLU 等非多项式激活函数同样成立。`https://doi.org/10.1016/S0893-6080(05)80131-5`

10. Tom B. Brown et al., *Language Models are Few-Shot Learners*, NeurIPS 2020。GPT-3 的层数、向量宽度、注意力头数、批次大小、参数量与训练计算量。https://arxiv.org/abs/2005.14165
11. Llama Team, AI @ Meta, *The Llama 3 Herd of Models*, 2024。模型层数、向量宽度、词表大小与训练数据量。https://arxiv.org/abs/2407.21783
12. Matus Telgarsky, *Benefits of Depth in Neural Networks*, COLT 2016。深层网络用少量神经元表示浅层网络需要指数级神经元的函数。https://arxiv.org/abs/1602.04485
13. Mor Geva et al., *Transformer Feed-Forward Layers Are Key-Value Memories*, EMNLP 2021。前馈层的“键-值记忆”解释，以及低层偏表层模式、高层偏语义模式。https://arxiv.org/abs/2012.14913
14. Jimmy Lei Ba, Jamie Ryan Kiros, Geoffrey E. Hinton, *Layer Normalization*, 2016。https://arxiv.org/abs/1607.06450
15. Biao Zhang, Rico Sennrich, *Root Mean Square Layer Normalization*, NeurIPS 2019。https://arxiv.org/abs/1910.07467
16. Ruibin Xiong et al., *On Layer Normalization in the Transformer Architecture*, ICML 2020。归一化放在子层之前（Pre-Norm）训练更稳定。https://arxiv.org/abs/2002.04745
17. Kaiming He et al., *Deep Residual Learning for Image Recognition*, CVPR 2016。深层网络的退化现象与残差连接。https://arxiv.org/abs/1512.03385
18. David E. Rumelhart, Geoffrey E. Hinton, Ronald J. Williams, *Learning Representations by Back-Propagating Errors*, Nature, 323, 533–536, 1986。https://doi.org/10.1038/323533a0
19. PyTorch 官方教程，*Optimizing Model Parameters*。损失、梯度、反向传播与参数更新。https://docs.pytorch.org/tutorials/beginner/basics/optimization_tutorial.html
20. Diederik P. Kingma, Jimmy Ba, *Adam: A Method for Stochastic Optimization*, ICLR 2015。https://arxiv.org/abs/1412.6980
21. Peter Grünwald, Teemu Roos, *Minimum Description Length Revisited*, 2019。从描述长度理解学习与模型选择。https://arxiv.org/abs/1908.08484
22. Yoshua Bengio et al., *A Neural Probabilistic Language Model*, JMLR 3, 2003。用词向量和神经网络缓解统计方法“没见过就查不到”的数据稀疏问题。https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf
23. Tomas Mikolov et al., *Efficient Estimation of Word Representations in Vector Space*, 2013。词向量及其类比关系。https://arxiv.org/abs/1301.3781
24. Nelson Elhage et al., *Toy Models of Superposition*, Transformer Circuits Thread, 2022。高维空间中“几乎垂直”的方向数量随维度指数增长（Johnson–Lindenstrauss 引理），以及模型如何用叠加表示多于维度数的特征。https://arxiv.org/abs/2209.10652
25. Ashish Vaswani et al., *Attention Is All You Need*, NeurIPS 2017。Transformer、缩放点积注意力、多头注意力与位置编码。https://arxiv.org/abs/1706.03762
26. Catherine Olsson et al., *In-context Learning and Induction Heads*, 2022。https://arxiv.org/abs/2209.11895
27. Jianlin Su et al., *RoFormer: Enhanced Transformer with Rotary Position Embedding*, 2021。旋转位置编码。https://arxiv.org/abs/2104.09864
28. Noam Shazeer, *GLU Variants Improve Transformer*, 2020。SwiGLU 等门控前馈网络。https://arxiv.org/abs/2002.05202
29. Ari Holtzman et al., *The Curious Case of Neural Text Degeneration*, ICLR 2020。Top-p（核采样），以及追求高概率导致文本重复、平淡的现象。https://arxiv.org/abs/1904.09751
30. Long Ouyang et al., *Training Language Models to Follow Instructions with Human Feedback*, NeurIPS 2022。InstructGPT、监督微调与 RLHF 的三步流程。https://arxiv.org/abs/2203.02155

31. Chunting Zhou et al., *LIMA: Less Is More for Alignment*, NeurIPS 2023。https://arxiv.org/abs/2305.11206
32. Rafael Rafailov et al., *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*, NeurIPS 2023。https://arxiv.org/abs/2305.18290
33. DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025。用可验证奖励的强化学习训练推理能力。https://arxiv.org/abs/2501.12948
34. Timo Schick et al., *Toolformer: Language Models Can Teach Themselves to Use Tools*, NeurIPS 2023。模型在文本中写出工具调用，由外部执行后把结果放回文本。https://arxiv.org/abs/2302.04761
35. Alexey Dosovitskiy et al., *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*, ICLR 2021。把图片切成小方块，当作 token 送进 Transformer。https://arxiv.org/abs/2010.11929
36. Jared Kaplan et al., *Scaling Laws for Neural Language Models*, 2020。规模定律与训练计算量估算。https://arxiv.org/abs/2001.08361
37. Jordan Hoffmann et al., *Training Compute-Optimal Large Language Models*, 2022。参数量与数据量的最优配比，以及包含不可约损失的拟合公式。https://arxiv.org/abs/2203.15556
38. Lei Huang et al., *A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions*, ACM TOIS。https://arxiv.org/abs/2311.05232
39. Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, Edwin Zhang, *Why Language Models Hallucinate*, 2025。零散事实与幻觉率的关系，以及评测机制如何鼓励“猜测”。https://arxiv.org/abs/2509.04664
40. Nelson F. Liu et al., *Lost in the Middle: How Language Models Use Long Contexts*, TACL 2024。https://arxiv.org/abs/2307.03172
41. Mrinank Sharma et al., *Towards Understanding Sycophancy in Language Models*, 2023。人类反馈训练与迎合行为。https://arxiv.org/abs/2310.13548
42. Lukas Berglund et al., *The Reversal Curse: LLMs Trained on “A is B” Fail to Learn “B is A”*, ICLR 2024。https://arxiv.org/abs/2309.12288
43. Alex Warstadt et al., *Call for Papers — The BabyLM Challenge: Sample-efficient Pretraining on a Developmentally Plausible Corpus*, 2023。儿童到 13 岁左右接触的词不到 1 亿个。https://arxiv.org/abs/2301.11796
44. David Silver et al., *Mastering the Game of Go without Human Knowledge*, Nature, 550, 354–359, 2017。AlphaGo Zero 不用人类棋谱，只靠自我对弈训练。https://doi.org/10.1038/nature24270





阅读 455

留言

写留言

还没有留言，来写首评